

SOCVerse

Software Architecture & Technical Design Specification

Document status: Draft v1.0 — Architecture Baseline

Prepared for: SOCVerse Engineering Team

Classification: Internal / Confidential

Date: 2026-07-28

Purpose of This Document

This document is the complete software architecture and technical design specification for SOCVerse, a proprietary, cloud-based cybersecurity training platform that simulates realistic Security Operations Center (SOC) investigations. It is written to be sufficiently detailed and unambiguous that an engineering team — or an AI coding agent — can implement the platform from an empty repository without having to make architectural assumptions. Every service boundary, data model, API contract, and infrastructure decision is intended to be treated as authoritative for implementation purposes unless a subsequent, explicitly versioned decision supersedes it.

SOCVerse is **not** a SIEM and depends on **no** existing SIEM, EDR, or identity platform (Wazuh, Splunk, Elastic, Sentinel, Chronicle, QRadar, or otherwise). All telemetry, detection logic, investigation surfaces, and scoring are proprietary, generated and owned entirely by this platform. See §1.6 for the explicit non-goals this implies.

Table of Contents

Software Architecture & Technical Design Specification.....	1
Table of Contents.....	3
1. Executive Summary.....	10
1.1 Product Vision.....	10
1.2 Why SOCVerse Exists	10
1.3 Market Opportunity.....	11
1.4 Target Users.....	11
1.5 Product Principles	12
1.6 Explicitly Not a SIEM	12
1.7 Commercial Vision	13
1.8 Attack Coverage at Launch	13
2. Functional Requirements.....	15
2.1 Alert Dashboard.....	15
2.2 Incident Queue	15
2.3 Case Management	16
2.4 Identity Portal	16
2.5 Device Portal.....	16
2.6 Email Portal.....	16
2.7 Threat Intelligence	17
2.8 Search	17
2.9 Global Timeline	17
2.10 Evidence Collection.....	18
2.11 Scenario Completion.....	18
2.12 Scoring	18
2.13 Analyst Notes.....	18
2.14 Evidence Collection (Reporting Output)	19
2.15 Instructor Mode.....	19
2.16 Admin Portal	19
2.17 Analytics.....	19
2.18 Certificates.....	20
2.19 Reporting	20
2.20 Investigation Actions (cross-cutting)	20
3. Non-Functional Requirements.....	21
3.1 Performance	21

3.2 Availability.....	21
3.3 Scalability	21
3.4 Latency (Real-Time Interaction).....	22
3.5 Security	22
3.6 Cloud Architecture	22
3.7 Maintainability	22
3.8 Reliability	23
3.9 Logging	23
3.10 Monitoring	23
3.11 Disaster Recovery	23
4. Complete System Architecture	25
4.1 High-Level Data Flow	25
4.2 Component Roles.....	25
4.3 Request Lifecycle Example (End-to-End)	27
4.4 Real-Time Path.....	27
4.5 Trust Boundaries.....	27
4.6 Why a "Modular Monolith" at MVP.....	28
5. Infrastructure Design	29
5.1 Technology Stack Summary	29
5.2 API Layer	29
5.3 Database Layer.....	30
5.4 Caching.....	30
5.5 Authentication & Authorization (Infrastructure View)	30
5.6 Data Access Abstraction	30
5.7 Authentication Token Details	31
5.8 Containerization.....	31
5.9 Message Queues & Background Workers	31
5.10 WebSockets	32
5.11 Object Storage	32
5.12 Scheduled Jobs.....	32
5.13 Deployment Architecture	32
5.14 CDN	33
5.15 Load Balancer.....	33
5.16 Monitoring & Logging (Infrastructure View).....	33
5.17 Secrets Management	33
5.18 Environment Variables.....	33

6. Database Design	34
6.1 Conventions & Tenancy Model	34
6.2 Entity-Relationship Overview	34
6.3 `users` and `roles`	35
6.4 `organizations`	36
6.5 `cohorts`	36
6.6 `cohort_enrollments`	37
6.7 `attack_scenarios`	37
6.8 `mitre_techniques`	38
6.9 `investigation_sessions`	38
6.10 `identities`	39
6.11 `devices`	40
6.12 Telemetry Event Tables	41
6.13 `investigation_actions`	43
6.14 `scores`	44
6.15 `alerts`	45
6.16 `incidents`	46
6.17 `evidence_collection` and `analyst_notes`	46
6.18 `certificates`	47
6.19 `refresh_tokens`	47
6.20 `instructor_feedback`	48
6.21 Learning Platform Tables	48
6.22 `audit_logs`	48
6.23 Data Retention & Partitioning Strategy	49
6.24 Migration Tooling & Change Management	50
7. Telemetry Generator	51
7.1 Purpose and Design Philosophy	51
7.2 Generation Pipeline	51
7.3 Realism Techniques Per Log Source	52
7.4 Volume and Difficulty Scaling	53
7.5 Randomization Without Losing Ground Truth	53
7.6 No Runtime LLM Dependency for Core Generation	53
7.7 Pre-Generation and Caching	53
8. Alert Engine	55
8.1 Purpose and Detection Model	55
8.2 Rule Categories	55

8.3 Correlation Logic.....	56
8.4 Alert Generation Timing	56
8.5 Severity Assignment.....	56
8.6 False Positives (By Design)	57
8.7 Deduplication and Suppression	57
8.8 Alert Prioritization on the Dashboard.....	57
9. Identity Investigation Service.....	58
9.1 Purpose	58
9.2 Identity Directory (List View)	58
9.3 Identity Profile	58
9.4 Sign-in Timeline.....	58
9.5 Risky Sign-ins.....	59
9.6 Impossible Travel Detection (Investigative Surface).....	59
9.7 Password Spray Detection (Investigative Surface)	59
9.8 Geo-location and Behavior Timeline	59
9.9 What Is Never Exposed to the Student.....	59
10. Device Investigation Service	61
10.1 Purpose	61
10.2 Device Overview	61
10.3 Process Tree.....	61
10.4 Network Connections and Open Ports	61
10.5 File Timeline.....	61
10.6 Registry Changes.....	62
10.7 USB Activity.....	62
10.8 Security Events (Device-Scoped Summary)	62
10.9 Malware History.....	62
10.10 Device Risk	62
10.11 Isolation Status and the Isolate Action	62
10.12 What Is Never Exposed to the Student.....	63
11. Email Investigation Module	64
11.1 Purpose	64
11.2 Mailbox/Message Search.....	64
11.3 Email Viewer	64
11.4 Authentication Results (SPF/DKIM/DMARC)	64
11.5 Attachment Analysis	64
11.6 URL Analysis	64

11.7 Sandbox Results	65
11.8 Campaign Correlation	65
11.9 Threat Intelligence Integration	65
11.10 What Is Never Exposed to the Student.....	65
12. Scenario Engine.....	66
12.1 Scenario Builder and Ground-Truth Definition Schema	66
12.2 Difficulty Levels.....	66
12.3 Hidden Ground Truth.....	67
12.4 Automated Evaluation (Scoring Engine)	67
12.5 Hints.....	68
12.6 Instructor Override and Scenario Versioning	68
12.7 Randomization and Live/Time-Phased Mode.....	68
13. Learning Platform	70
13.1 Courses and Labs	70
13.2 Progress Tracking.....	70
13.3 Achievements	70
13.4 Certificates.....	70
13.5 Leaderboards	70
13.6 Learning Paths and Career Tracks.....	71
13.7 Instructor-Facing Learning Platform Features	71
14. AI Features	72
14.1 Scope and Positioning.....	72
14.2 AI Investigation Assistant.....	72
14.3 AI Hint System.....	72
14.4 Natural Language Search	72
14.5 AI Report Writer.....	73
14.6 AI Feedback.....	73
14.7 SOC Copilot (Longer-Term Vision)	73
14.8 Cost and Abuse Controls.....	73
15. Security Architecture	75
15.1 Authentication	75
15.2 Authorization (RBAC)	75
15.3 Encryption.....	75
15.4 Secrets Management.....	76
15.5 Rate Limiting	76
15.6 API Security	76

15.7 Session Management.....	77
15.8 Payment Security	77
15.9 Audit Logging	77
15.10 Input Validation	77
15.11 Secure Coding Practices	78
15.12 Data Subject Rights / Account Deletion.....	78
16. API Specification	79
16.1 Conventions	79
16.2 Auth Service (`/api/v1/auth`).....	79
16.3 Scenario Catalog Service (`/api/v1/scenarios`).....	79
16.4 Session Service (`/api/v1/sessions`).....	80
16.5 Alert Service (`/api/v1/sessions/{sessionId}/alerts`)	80
16.6 Incident Service (`/api/v1/sessions/{sessionId}/incidents`)	80
16.7 Evidence, Notes, and Timeline (`/api/v1/sessions/{sessionId}/incidents/{incidentId}`)	81
16.8 Investigation Actions (`/api/v1/sessions/{sessionId}/actions`)	81
16.9 Identity Portal (`/api/v1/sessions/{sessionId}/identities`)	81
16.10 Device Portal (`/api/v1/sessions/{sessionId}/devices`)	81
16.11 Email Portal (`/api/v1/sessions/{sessionId}/emails`).....	82
16.12 Search and Threat Intel (`/api/v1/sessions/{sessionId}`).....	82
16.13 Instructor Service (`/api/v1/instructor`)	82
16.14 Admin Service (`/api/v1/admin`)	82
16.15 Learning Platform Service (`/api/v1/learning`).....	83
16.16 WebSocket Channel.....	83
17. Frontend Architecture	84
17.1 Application Shells.....	84
17.2 Routing.....	84
17.3 State Management	84
17.4 Component Library and Design System	85
17.5 Key Pages and Navigation Flow	85
17.6 API Client Layer	85
17.7 Responsive Behavior	86
17.8 Accessibility.....	86
17.9 Real-Time and Offline-Degradation Behavior.....	86
17.10 Performance Budget.....	86
18. Backend Architecture.....	87
18.1 Service Inventory	87

18.2 Identity/Auth Service — Internal Workflow Example	88
18.3 Cross-Cutting: The Serialization/DTO Layer.....	88
18.4 Session Service — Orchestration Workflow	88
18.5 Learning Platform Service — Progress Computation.....	88
18.6 Error Handling Strategy.....	89
18.7 Scheduling and Scaling Strategy Per Service.....	89
19. Deployment Strategy	91
19.1 Phase 1: Single VPS (MVP)	91
19.2 Phase 1 Sizing.....	91
19.3 Phase 2: Managed Data Services, Same Compute Topology	91
19.4 Phase 3: Kubernetes, Autoscaling, High Availability	92
19.5 Disaster Recovery Runbook (Summary)	92
19.6 CDN and Static Asset Delivery Across Phases	92
19.7 Environment Parity	93
20. Cost Optimization	94
20.1 Guiding Principle	94
20.2 Cost Breakdown by Category (Phase 1 / MVP).....	94
20.3 Deliberate Cost/Quality Trade-offs Made Explicit	95
20.4 What Triggers Each Spend Increase.....	96

1. Executive Summary

1.1 Product Vision

SOCVerse is a cloud-native, proprietary cybersecurity training platform that reproduces the day-to-day experience of working inside a modern Security Operations Center (SOC). Where existing training products rely on video lectures, multiple-choice quizzes, or static "capture the flag" challenges, SOCVerse instead places the learner inside a fully interactive, purpose-built investigation environment that looks, feels, and behaves like the enterprise security tooling professionals use in production: an alert queue, an incident workbench, an identity investigation console modeled conceptually on Microsoft Entra ID, an endpoint console modeled conceptually on Microsoft Defender for Endpoint, and an email investigation console modeled conceptually on Microsoft Defender for Office 365 / Outlook.

Critically, SOCVerse is **not** a SIEM, and it does not ingest, store, or process any real customer telemetry. It is a closed-loop educational simulator: the platform itself generates synthetic but statistically and behaviorally realistic telemetry (sign-in logs, process trees, DNS queries, email headers, firewall events, etc.), embeds a hidden "ground truth" attack narrative inside that telemetry, and then challenges the learner to reconstruct that narrative through investigation — exactly as a real analyst would reconstruct an attacker's actions from log data. The backend already knows the answer. The learner's job is to find it, document it, and defend their conclusion. This is the same operating model as a flight simulator: the aircraft never actually leaves the ground, but the controls, the failure conditions, and the consequences of a wrong decision are real enough to build genuine muscle memory and judgment.

This document is the full software architecture and technical design specification for SOCVerse. It is written so that an engineering team — or an AI coding agent — can implement the platform from an empty repository without having to make architectural guesses. Every service, data model, API contract, and infrastructure decision described here is intended to be treated as authoritative unless a future decision explicitly supersedes it.

1.2 Why SOCVerse Exists

The cybersecurity industry has a well-documented and persistent workforce gap: industry estimates have placed the global shortage of security professionals at several million unfilled roles for multiple consecutive years, and the entry-level SOC analyst (Tier 1) role is simultaneously the highest-turnover and highest-demand position in the field. The core problem is not a lack of course material — there is no shortage of certifications, video courses, or written material explaining what phishing, lateral movement, or impossible travel are in the abstract. The problem is that almost none of that material gives a learner hands-on repetitions of the actual *investigative workflow*: opening an alert, pivoting from a sign-in event to a device, from a device to a process tree, from a process tree to a file hash, correlating that hash against threat intelligence, and writing up a defensible incident conclusion under time pressure and incomplete information.

Existing hands-on options fall short in specific, addressable ways:

- **Real SIEM trial/sandbox environments** (e.g., a Sentinel or Splunk trial workspace) require the learner to also learn a general-purpose, enterprise-grade query language and administrative console before they

can practice investigation skills at all. They are also expensive to operate at scale for a training vendor, since real SIEM licensing and compute costs scale with data ingested.

- **Capture-the-flag (CTF) platforms** teach exploitation and offensive skills, not defensive triage and investigation. The "find the flag" objective structure does not match the "determine whether this is malicious, and if so, what happened" objective structure of SOC work.
- **Video-based certification prep** builds vocabulary and conceptual knowledge but produces no evidence that the learner can actually perform an investigation, because there is no interactive investigation to perform.
- **Static screenshot-based walkthroughs** cannot adapt, cannot be graded programmatically, and cannot vary between attempts, so they are trivially memorized rather than learned.

SOCVerse is designed to fill exactly this gap: a scenario-driven, professionally faithful, infinitely repeatable investigation simulator, delivered as a subscription SaaS product, that can plausibly replace (or meaningfully supplement) the hands-on-labs component of SOC analyst training programs, university cybersecurity programs, corporate blue-team upskilling programs, and individual certification study plans.

1.3 Market Opportunity

The addressable market spans several buyer segments, each with a distinct purchasing motion:

Segment	Buyer	Motion	Notes
Individual learners / career changers	Self-pay, monthly or annual subscription	Direct-to-consumer, content marketing + SEO + community	Comparable in spirit to TryHackMe/LetsDefend pricing bands
Bootcamps & universities	Program directors, department budgets	Seat-based licensing, per-cohort pricing	Requires Instructor Mode, class rosters, grading exports
Corporate security teams	SOC managers, L&D budgets	Team seats, SSO, usage reporting for compliance training credit	Requires SSO, audit logs, admin reporting
MSSPs training new analysts	Ops managers	Bulk seats tied to onboarding pipelines	Requires fast onboarding, scenario assignment automation

Direct competitors and adjacent products (LetsDefend, TryHackMe's SOC-Level content, CyberDefenders, RangeForce, Immersive Labs) validate demand for this category but each has gaps SOCVerse is designed to exploit: most are narrowly focused on a single investigation surface (usually just a log search interface), few offer a multi-console experience spanning identity, endpoint, and email simultaneously the way a real Tier 1/Tier 2 analyst's day actually spans those surfaces, and few are built with a scenario engine flexible enough to support instructor-authored custom content — which is the feature that unlocks the B2B education and enterprise segments, not just the B2C segment.

1.4 Target Users

- **Aspiring SOC analysts** with little to no professional experience, using SOCVerse to build a portfolio of demonstrable investigation reps before applying for Tier 1 roles.
- **Junior analysts (0–2 years)** looking to accelerate past Tier 1 pattern-matching into genuine investigative reasoning, and to practice attack types their current employer hasn't yet exposed them to (e.g., BEC, ransomware precursors).
- **Career changers** (IT support, sysadmins, military/veterans transitioning to cyber) who have adjacent technical literacy but no security-specific investigative experience.
- **University and bootcamp instructors** who need a gradable, reusable lab environment instead of building one-off labs by hand each term.
- **Corporate blue teams** running internal tabletop and upskilling exercises, particularly around a new attack technique or after a real incident post-mortem ("let's make sure the whole team can recognize this pattern").

1.5 Product Principles

1. **Fidelity without dependency.** The platform must *feel* like Sentinel/Defender/Splunk in its investigative ergonomics — because that ergonomic fluency is the actual skill being taught — while remaining 100% proprietary in implementation. No vendor SIEM, EDR, or identity product is embedded, wrapped, or resold. See §1.6.
2. **Ground truth first.** Every scenario is authored backwards from a known attacker narrative (a MITRE ATT&CK-mapped kill chain) into synthetic telemetry, never forwards from telemetry into an ambiguous guess. This is what makes automated, objective grading possible (§12).
3. **Investigation, not search-box trivia.** The grading model rewards correct *conclusions* reached via a defensible evidence trail, not merely finding a keyword. Partial credit, hint costs, and false-positive penalties are first-class scoring concepts (§6.14, §12).
4. **Cost-conscious by default.** As a bootstrapped/early-stage product, every infrastructure choice defaults to the cheapest option that does not compromise the learning experience, with an explicit, pre-planned upgrade path as revenue grows (§19, §20).
5. **Content is the moat.** The scenario library, the realism of the synthetic telemetry, and the instructor tooling are the durable competitive advantages — not any single UI screen. Architecture decisions throughout this document are made to keep scenario authoring cheap and scenario variety high.

1.6 Explicitly Not a SIEM

This distinction is architecturally load-bearing and is repeated here because it shapes almost every downstream decision in this document:

- SOCVerse does not accept customer-supplied log data of any kind. There is no "connect your data source" feature. All telemetry is synthetic, generated by the platform itself (§7).
- SOCVerse does not implement a general-purpose query language, a rules-as-code detection authoring surface for end users, or a data-ingestion pipeline sized for real enterprise log volume. The "Search" and

"Global Timeline" features (§2) are investigation conveniences scoped to a single scenario's dataset (typically thousands, not billions, of events), not a big-data analytics product.

- SOCVerse's "Identity Portal," "Device Portal," and "Email Portal" are **inspired by** the information architecture of Entra ID, Defender for Endpoint, and Outlook/Defender for Office 365 so that the investigative *muscle memory* transfers to real tools — but every field, screen, and API in this document is an original design, backed by SOCVerse's own database schema (§6), not a clone of any vendor's UI, code, or protected trade dress.

1.7 Commercial Vision

Phase 1 (MVP, months 0–6): Single-tenant-per-user SaaS, individual subscriptions only, a curated library of 15–25 hand-authored scenarios spanning the attack categories in §1.8, delivered on the lean single-VPS deployment described in §19. Goal: prove that the investigation experience is compelling enough to retain paying individual subscribers month over month, and start generating the usage data needed to tune the scoring engine.

Phase 2 (months 6–14): Instructor Mode, classroom/cohort seat licensing, scenario assignment and grading export, SSO for institutional buyers, and a self-service scenario authoring tool so power users and partner instructors can contribute scenarios (curated, not fully open marketplace yet). Infrastructure migrates toward the managed-Postgres/Kubernetes-readiness path described in §19.3.

Phase 3 (14+ months): Enterprise team seats with admin reporting suitable for compliance/training-credit purposes, AI Investigation Assistant and AI-authored scenario variation (§14) to scale content production, and a scenario marketplace with revenue share for community-authored content. Full Kubernetes, multi-region, and HA posture (§19.4) is justified by this point by actual paid load.

Revenue model: monthly/annual subscription tiers for individuals (Free trial → Pro), per-seat annual licensing for institutions/enterprises, and an optional one-time "certification exam attempt" product (a proctored, higher-stakes scenario battery) as a longer-term upsell once the core product and content library are proven.

1.8 Attack Coverage at Launch

The scenario library (§12) must, at MVP, include working, gradable scenarios for each of the following investigation domains, each mapped to relevant MITRE ATT&CK techniques (§6, `mitre_techniques` table):

- Microsoft Entra ID–style identity investigations (conditional access anomalies, risky sign-ins)
- Microsoft Defender–style endpoint investigations (process trees, persistence artifacts)
- Email phishing investigations (headers, SPF/DKIM/DMARC, malicious attachments/URLs)
- General endpoint investigations (malware execution, USB exfiltration)
- Authentication investigations (credential misuse, session hijacking)
- Cloud investigations (IaaS/SaaS control-plane misuse, storage exposure)
- Insider threat investigations (anomalous data access by legitimate credentials)
- Web attack investigations (web shell deployment, injection-driven compromise)

- Malware investigations (execution chain, C2 beaconing)
- Ransomware investigations (encryption precursors, lateral spread, exfil-before-encrypt)
- Lateral movement (MITRE TA0008)
- Persistence (MITRE TA0003)
- Privilege escalation (MITRE TA0004)
- Impossible travel
- Password spraying
- MFA bypass / MFA fatigue
- Business Email Compromise (BEC)
- Data exfiltration

Each of these is not necessarily a separate scenario — a well-authored ransomware scenario, for example, naturally exercises initial access, privilege escalation, lateral movement, and persistence in one narrative — but each must be independently *assessable*, meaning the scoring engine (§6.14, §12.4) must be able to tell whether the learner correctly identified each technique present, not just whether they reached the final "this was ransomware" verdict.

2. Functional Requirements

This section enumerates every user-facing feature at MVP scope, its purpose, its primary actors, and its acceptance criteria at a level detailed enough to derive UI wireframes and API contracts from. Deeper technical treatment of each feature's backend service lives in §8–§14 and §18; deeper UI treatment lives in §17.

Actors used throughout this document:

- **Student** — a learner working scenarios, either self-enrolled or assigned by an instructor.
- **Instructor** — creates/assigns scenarios, reviews student work, issues feedback and overrides.
- **Org Admin** — manages an institutional/enterprise tenant: seats, SSO, billing, reporting.
- **Platform Admin** — SOCVerse operator staff; manages scenario library, global config, support.
- **System** — automated actors: Telemetry Generator, Alert Engine, Scoring Engine.

2.1 Alert Dashboard

Purpose: The first screen a Student sees when a scenario session begins; mirrors the "alerts queue" of a real SIEM/XDR console. Presents all alerts the Alert Engine (§8) has generated for the active `investigation_session`.

Requirements:

- List view with columns: severity, title, entity (identity/device/mailbox), MITRE technique tag(s), status (New / In Progress / Resolved / Dismissed), first-seen timestamp, alert count if deduplicated (§8.7).
- Filter/sort by severity, status, entity type, technique, time range.
- Clicking an alert opens the Incident/Alert detail view, showing the raw evidence that triggered it (the specific `security_events` rows referenced by the alert's `alert_evidence_refs`) and a "Promote to Incident" or "Add to Incident" action.
- Bulk actions: assign multiple alerts to a new or existing incident, bulk-dismiss with a required dismissal reason (feeds §8.9 false-positive tracking and scoring).
- Real-time updates via WebSocket (§5) as the Telemetry/Alert Engine emits new alerts mid-session for scenarios with a time-phased narrative (§12.7).

Acceptance criteria: A Student can, without instructor help, understand which alerts exist, drill into the evidence behind any one alert, and either dismiss it or escalate it into a case, all without leaving the dashboard route.

2.2 Incident Queue

Purpose: Case-manager-style list of `incidents` — the unit of work a Student is actually graded on (an alert is raw signal; an incident is the Student's claim about what happened).

Requirements:

- List of incidents in the current session with status (Open / Investigating / Contained / Closed), severity, assigned analyst (self, in solo mode), linked alert count, linked entity count, SLA/time-in-status indicator.
- Create incident manually (Student-initiated) or via "Promote to Incident" from the Alert Dashboard.
- Incident detail view aggregates: linked alerts, linked entities (identities/devices/mailboxes), the Global Timeline scoped to this incident, Analyst Notes, Evidence Collection items, and the final Verdict submission form (§2.12).

Acceptance criteria: A Student can build a single incident out of multiple related alerts (e.g., a risky sign-in alert + a malware-execution alert on the device that signed in), and the platform persists that grouping distinctly from the underlying alerts.

2.3 Case Management

Purpose: The workspace for a single incident's lifecycle: status transitions, ownership, SLA tracking, and closure workflow (verdict + classification + MITRE technique tagging + final report).

Requirements:

- Status state machine: Open → Investigating → Contained → Closed with Reopened as a valid transition from Closed. Each transition is timestamped and stored in `investigation_actions` for scoring/replay (§6.13, §2.20).
- Closure requires: a verdict (True Positive / False Positive / Benign Positive), a written summary (free text, minimum length enforced), and technique tags selected from `mitre_techniques`.
- Instructor override capability (§2.15): an instructor can reopen, re-classify, or annotate a closed case without altering the Student's original submission (stored as a separate `instructor_feedback` record, §6.20).

Acceptance criteria: Case status and verdict are immutable once submitted except via an explicit, audited instructor reopen action.

2.4 Identity Portal

Full detail in §9. Functional summary: searchable directory of `identities` in the active scenario; per-identity profile with risk level, MFA status, group membership, manager, devices, and a sign-in timeline with geo/risk annotations (impossible travel, password spray correlation) rendered as investigable evidence, not simply displayed as an answer key.

2.5 Device Portal

Full detail in §10. Functional summary: searchable directory of `devices`; per-device profile with process tree, installed software, network connections, file timeline, registry changes (Windows-flavored scenarios), USB activity, and isolation action (a scorable containment action, §2.20).

2.6 Email Portal

Full detail in §11. Functional summary: mailbox/message search; message reader with raw header view, SPF/DKIM/DMARC results, attachment and URL analysis panels, and campaign correlation across other messages in the same scenario dataset.

2.7 Threat Intelligence

Purpose: A proprietary, scenario-scoped threat intel lookup surface, avoiding any dependency on real external TI feeds (which would leak real-world answers or introduce non-reproducible results).

Requirements:

- `threat_intel_indicators` table (§6) seeded per-scenario with IOCs (hashes, IPs, domains, URLs) that are relevant to that scenario's ground truth, plus deliberate decoy/benign indicators to avoid trivial keyword-matching solutions.
- Lookup panel accessible from Device Portal (file hash), Email Portal (URL/domain), and a standalone search page.
- Each indicator carries a synthetic "reputation" (Malicious / Suspicious / Unknown / Known-good), a synthetic actor/campaign attribution where relevant, and free-text context — modeled on the information density of a real TI platform response, without querying any live external service.

Acceptance criteria: Every IOC a Student needs to resolve a scenario is resolvable via this in-platform lookup; no scenario ever requires an external internet lookup.

2.8 Search

Purpose: Cross-entity keyword and structured search scoped to the active session's synthetic dataset — the SOCVerse analog of a SIEM search bar, deliberately simplified (§1.6).

Requirements:

- Structured filter builder (field = value, with AND) covering the common investigable fields (source IP, user, device, process name, file hash, URL, sender) across `security_events`.
- Freetext search over event raw fields.
- Results are timestamped, entity-tagged, and each result can be added directly to Evidence Collection (§2.10) or the Global Timeline (§2.9) with one action.

Acceptance criteria: Search never requires knowledge of a proprietary query language beyond field=value pairs; results appear in under 500ms P95 for a session-scoped dataset (§3.1).

2.9 Global Timeline

Purpose: A unified, chronologically sorted view of every event a Student has pulled in as relevant (from Search, Identity Portal, Device Portal, Email Portal), used to visually reconstruct the attack chain.

Requirements:

- Drag-free, click-to-add model: any evidence item, anywhere in the product, has an "Add to Timeline" action.

- Timeline view groups by entity/lane (per-identity, per-device, per-mailbox swimlanes) with cross-entity correlation lines where the platform can determine two events share a session/correlation ID (§6.9).
- Exportable as part of the final incident report (§2.12).

Acceptance criteria: A Student can construct an ordered narrative of the incident purely from items they've curated onto the timeline, and that curated set is itself part of what the Scoring Engine evaluates (did they include the right events; did they include noise).

2.10 Evidence Collection

Purpose: A scoped "evidence locker" per incident — distinct from the Timeline (which is about *order*) in that Evidence Collection is about *relevance/proof*.

Requirements:

- Any record (event, alert, identity attribute, device artifact, email) can be pinned to Evidence Collection with an analyst-written justification note.
- Evidence items are individually taggable to a specific MITRE technique.
- Evidence Collection is a required, scored input to the final incident report (§2.12, §12.4).

2.11 Scenario Completion

Purpose: The end-state flow when a Student submits a final verdict on an incident (or set of incidents) tied to a scenario.

Requirements:

- Submission requires: verdict, technique tags, written summary, and a minimum evidence-collection count (configurable per scenario).
- On submission, session locks (no further evidence changes) and is handed to the Scoring Engine (§12.4) for immediate automated scoring, with a results screen showing score breakdown, missed evidence, and false positives flagged.
- Optional "Request Instructor Review" flag for cohort-assigned scenarios, queuing it in the Instructor Mode review list (§2.15).

2.12 Scoring

Full detail in §12.4 and §6 (scores, investigation_sessions). Functional summary: composite score built from technique-identification accuracy, evidence precision/recall, false-positive rate, hint usage penalty, and time-to-resolution, displayed as an overall percentage plus a rubric breakdown, never as an opaque single number alone.

2.13 Analyst Notes

Purpose: Freeform, timestamped note-taking scoped to an incident, for the Student's own working memory — not directly graded, but retained and visible to instructors and to the Student in their own review/replay.

Requirements: Rich-text (constrained: bold/italic/lists/code-inline only — no arbitrary HTML) notes, autosaved, versioned by timestamp, visible in the incident detail sidebar at all times.

2.14 Evidence Collection (Reporting Output)

Covered by §2.10/§2.11; the "final report" artifact assembled from Notes + Evidence Collection + Timeline + Verdict is rendered as a structured document (HTML in-app, exportable to PDF) — see §2.19 Reporting.

2.15 Instructor Mode

Purpose: Enables the B2B/education commercial motion (§1.7 Phase 2).

Requirements:

- Class/cohort creation and roster management (invite by email or join code).
- Scenario assignment to a cohort with a due date and attempt limit.
- Live review queue: submitted sessions awaiting instructor sign-off, with side-by-side view of Student evidence trail vs. ground truth.
- Instructor Feedback: free-text + rubric-item overrides, stored distinctly from the automated score (`instructor_feedback` table, §6.20) so automated and human assessment are both preserved.
- Gradebook export (CSV) per cohort per scenario.
- Instructor-authored scenario creation entry point (delegates to Scenario Builder, §12.1), gated to Instructor/Admin roles.

2.16 Admin Portal

Purpose: Org Admin and Platform Admin operations surface, separate application shell from the Student/Instructor experience (§17.1).

Requirements (Org Admin scope): seat management, SSO configuration, billing/plan view, usage reporting across the org's cohorts.

Requirements (Platform Admin scope): scenario library management (publish/unpublish/version), global user search and support actions (impersonation with full audit logging, §15.10), platform-wide analytics (§2.17), feature flag configuration, incident-response tooling for the platform itself (not to be confused with in-scenario incidents).

2.17 Analytics

Purpose: Both learner-facing progress analytics and org/platform-facing usage analytics.

Requirements:

- Student-facing: per-skill radar/trend (accuracy by MITRE tactic over time), scenario completion history, time-on-task trends, leaderboard position (§13.5).

- Org-facing: cohort completion rates, average score by scenario, time-to-completion distributions, skill-gap heatmap across a cohort (which techniques does this cohort systematically miss).
- Platform-facing: DAU/MAU, funnel from signup → first scenario completion → subscription conversion, scenario-level difficulty calibration data (is a scenario's real-world pass rate matching its authored difficulty label) feeding back into §12.1.

2.18 Certificates

Purpose: Shareable proof of completion for learning paths (§13.6) and, longer-term, the proctored certification product (§1.7 Phase 3).

Requirements: Auto-issued PDF/verifiable-link certificate on learning-path completion meeting a minimum score threshold; unique verification URL (`/verify/{certificate_uuid}`) that renders a public, minimal-PII confirmation page.

2.19 Reporting

Purpose: Structured incident report generation (Student-facing, per §2.11/§2.14) and org-facing usage/compliance reporting (Admin-facing, per §2.16/§2.17), both exportable as PDF/CSV.

2.20 Investigation Actions (cross-cutting)

Certain in-product actions are themselves scorable "response" behaviors, not just information-gathering:

Isolate Device, Disable Account/Force Password Reset, Block Sender/Domain, Escalate to Incident, Dismiss Alert (with reason). Every such action is recorded in `investigation_actions` (§6.13) with actor, timestamp, and target entity, both to support the Global Timeline/replay feature and because a scenario's rubric may require (or penalize) specific containment actions (e.g., "did the Student isolate the compromised host before or after further lateral movement occurred in the simulated timeline").

3. Non-Functional Requirements

Non-functional requirements are stated as concrete, testable targets wherever possible, tiered by deployment phase (§19) since the acceptable numbers for a single-VPS MVP are intentionally looser than for the post-PMF Kubernetes deployment. Where a target changes between phases, both are given as **MVP / Scale**.

3.1 Performance

- API P95 latency for read endpoints (alert list, entity profile, timeline fetch): **< 300ms MVP / < 150ms Scale**, measured server-side, excluding client network time.
- API P95 latency for write endpoints (note save, evidence pin, verdict submit): **< 500ms MVP / < 250ms Scale**.
- Search endpoint (§2.8) P95: **< 500ms MVP / < 200ms Scale** for a session-scoped dataset capped at 250,000 synthetic events (§3.9).
- WebSocket event delivery (new alert push, §5.10): **< 2s from Alert Engine emission to client receipt, both phases**.
- Scenario session cold-start (telemetry generation + alert seeding, §7/§8) must complete in **< 20s MVP / < 8s Scale**, because this is a first-impression latency a paying Student experiences every time they start a scenario. This directly drives the "pre-generate at publish time, not at session start" design decision in §7.7.
- Frontend initial load (Time to Interactive) on the Student dashboard: **< 2.5s on a broadband connection**, driven by the code-splitting strategy in §17.

3.2 Availability

- MVP target: **99.0% monthly** (≈ 7.3 hours downtime/month budget), consistent with a single-region, single-primary-database deployment on a VPS (§19.1). This is stated honestly rather than aspirationally — a bootstrapped single-VPS deployment cannot credibly promise "four nines," and promising it would create a support/refund liability. Maintenance windows are scheduled and communicated, not counted against the budget separately (i.e., the 99.0% figure already assumes competent, low-frequency maintenance).
- Scale target (post-Kubernetes migration, §19.4): **99.9% monthly** (≈ 43 minutes/month), backed by multi-AZ database (managed Postgres with standby), horizontally scaled stateless API pods, and a load balancer health-check-driven failover.
- The platform is **not** targeting 99.99%+ availability at any phase in this document's horizon — that tier is not commercially justified for an education product (unlike, say, a real SOC's actual SIEM, where downtime has security consequences). This is a deliberate, cost-driven scoping decision (§20).

3.3 Scalability

- MVP must comfortably support **500 concurrent active scenario sessions** on the single-VPS topology (§19.1) sized in §19.2, with headroom validated by load testing before general-availability launch.

- The architecture must scale horizontally by adding stateless API/worker replicas behind a load balancer with **no code changes**, only infrastructure changes — this is why session state, WebSocket presence, and job queues are designed from day one to live in Redis/Postgres rather than in-process memory (§5, §18).
- Telemetry generation and scoring are CPU/IO-bound background jobs, not request-path work, and are explicitly designed as horizontally scalable worker pools (§5.9, §18) so that scenario-session growth scales by adding workers, independent of API replica count.
- Database scalability: partitioning strategy for high-volume tables (`security_events`, `audit_logs`) is specified up front in §6.23 even though MVP volume does not require it yet, so the migration to partitioned tables at scale is a config change, not a schema redesign.

3.4 Latency (Real-Time Interaction)

- WebSocket connection establishment: **< 1s**.
- Scenario "live" mode (time-phased alert emission, §12.7) must maintain event ordering and deliver events within **± 2s** of their scheduled scenario-relative timestamp — timing precision matters pedagogically (a Student reasoning about "what happened first" must be able to trust the platform's delivery order).

3.5 Security

Full treatment in §15. Summary requirements gating all other design: TLS 1.2+ everywhere (no plaintext internal traffic across any network boundary that crosses a host, §15.3), argon2id password hashing, short-lived JWT access tokens with rotating refresh tokens, RBAC enforced at the API-gateway layer and re-validated at the service layer (defense in depth, §15.2), encryption at rest for the database volume and object storage, secrets never committed to source control or baked into images (§5.16), full audit logging of privileged actions (§15.9), and rate limiting on all public-facing endpoints (§15.5).

3.6 Cloud Architecture

The platform must be deployable, without code changes, on any of: a single Linux VPS via Docker Compose (MVP default, §19.1), or a managed Kubernetes service on any major cloud provider (§19.4). This portability requirement is why the architecture avoids any cloud-provider-proprietary managed service in its core dependency graph (Postgres, Redis, S3-compatible object storage, and a generic message queue are all available as both self-hosted OSS and managed-cloud offerings, §5).

3.7 Maintainability

- Backend services are organized as independently deployable modules with clear API boundaries (§18) even in the MVP's "modular monolith" deployment topology (§4.6), so that splitting any module into its own microservice later is a deployment change, not a rewrite.
- All services expose structured JSON logs (§5.14) and a `/health` and `/ready` endpoint.
- Database schema changes are managed exclusively through versioned, forward-only migrations (no manual schema edits against production, §6.24).

- Configuration is environment-variable-driven (§5.17) with no environment-specific code branches.

3.8 Reliability

- All background jobs (telemetry generation, scoring, email sending, certificate generation) are processed through a durable message queue with at-least-once delivery and idempotent job handlers (§5.9), so a worker crash mid-job never silently loses or double-applies work.
- Database writes for anything score-affecting (verdict submission, evidence pinning) occur inside transactions; partial writes must not be observable.
- Graceful degradation: if the WebSocket layer is unavailable, the frontend falls back to polling for alert updates rather than failing the session outright (§17.9).

3.9 Logging

- Application logs: structured JSON, correlation-ID-tagged per request (propagated from API Gateway through every downstream service call, §5.2), shipped to a centralized log store (self-hosted Loki in MVP, §19.1; managed log service optional at Scale).
- Domain audit logs (§6.22 `audit_logs` table) are logically distinct from application/operational logs: audit logs are a permanent, tamper-evident record of security-relevant and grading-relevant actions (login, role change, verdict submission, instructor override, admin impersonation), retained per the retention policy in §6.23, and are never subject to log-rotation deletion the way operational logs are.
- Synthetic security event volume per session is capped (default 50,000–250,000 events per scenario dataset, tunable per scenario) specifically so that "logging" in the pedagogical sense (the telemetry the Student investigates) never blurs with or overwhelms real application/audit logging.

3.10 Monitoring

- Golden signals (latency, traffic, errors, saturation) collected per service via Prometheus-compatible metrics exposition (§5.15), visualized in a self-hosted Grafana instance at MVP.
- Alerting on: API error-rate threshold, queue depth/age threshold (a stuck worker pool is a direct product-quality incident, since it delays scenario start), database connection saturation, disk usage threshold on the VPS/object storage.
- Business-metric monitoring (signup rate, conversion rate, churn) is a separate, product-analytics concern (§2.17) from infrastructure monitoring, but both dashboards are considered part of "launch readiness" for any release.

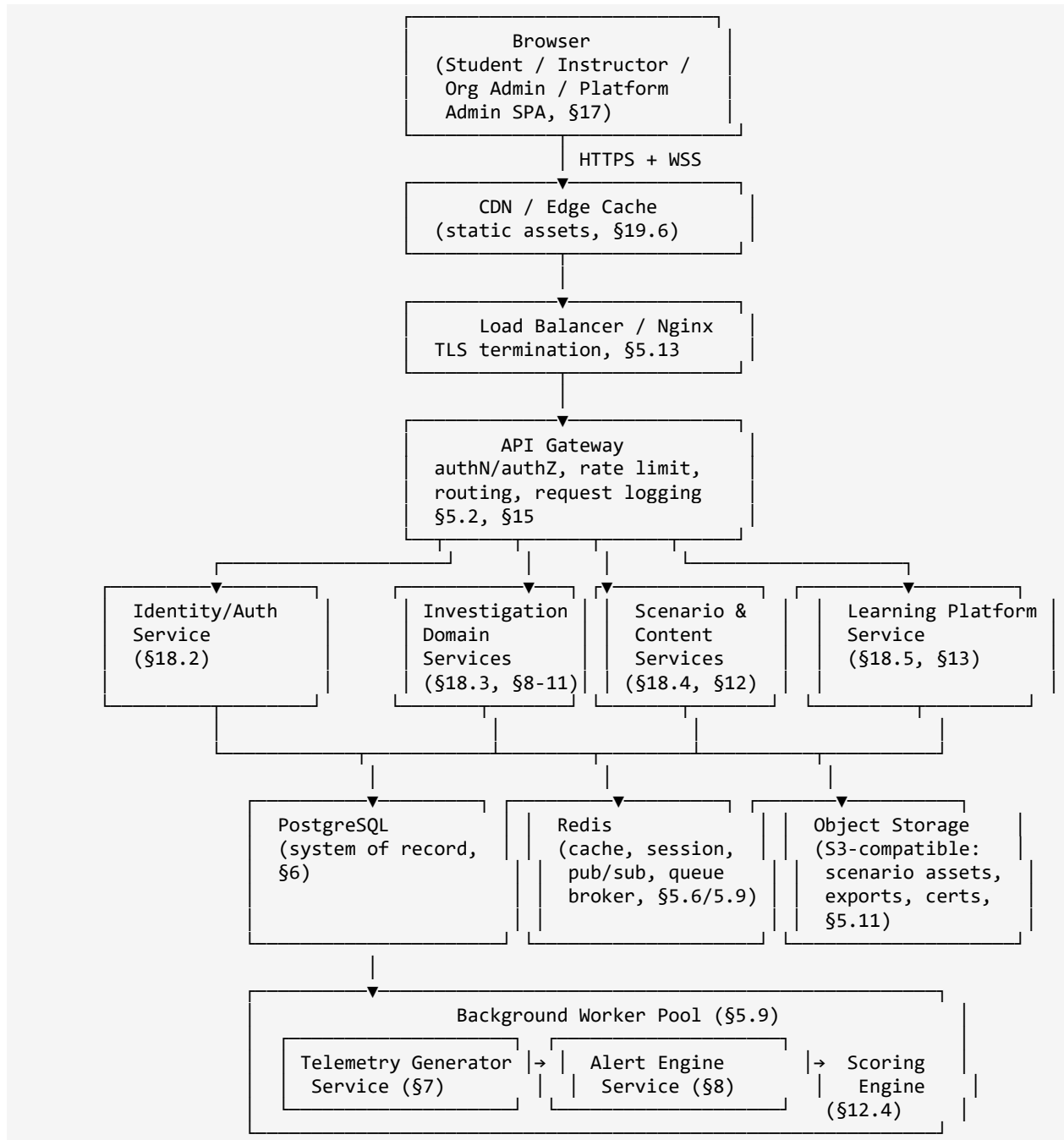
3.11 Disaster Recovery

- **RPO (Recovery Point Objective): MVP 24 hours / Scale 5 minutes.** MVP relies on nightly automated Postgres logical backups (`pg_dump`) plus continuous WAL archiving to object storage, which in practice allows point-in-time recovery closer to minutes, but 24h is the conservative committed SLA until this is proven in a real restore drill. Scale phase moves to a managed database with continuous backup and point-in-time restore as a vendor-provided guarantee.

- **RTO (Recovery Time Objective): MVP 4 hours / Scale 30 minutes.** A documented, periodically-drilled runbook (§19.5) covers full-stack restore from backup onto a fresh VPS.
- Object storage (scenario assets, uploaded evidence exports, certificates) is versioned and separately backed up from the primary database (§5.11), since a bug that corrupts one must not be able to corrupt the other in the same failure event.
- Backups are stored in a provider/region distinct from the primary compute host, satisfying the basic "backup is not co-located with the thing it backs up" requirement even at MVP cost levels (§20).

4. Complete System Architecture

4.1 High-Level Data Flow



4.2 Component Roles

Browser (SPA). A single frontend codebase serving four role-scoped shells (Student, Instructor, Org Admin, Platform Admin — §17.1) from one build, route-gated by role after authentication. Communicates with the

backend exclusively through the API Gateway's REST endpoints (§16) and a single WebSocket channel per active session for real-time alert/timeline push (§5.10).

CDN / Edge Cache. Serves the built frontend bundle, public marketing pages, and public certificate-verification pages. At MVP this is a lightweight, low/no-cost CDN in front of the static asset bucket; it is not in the critical path for any authenticated API traffic and is the first component to swap for a bigger provider as traffic grows, with zero backend impact (§20.2).

Load Balancer / Reverse Proxy (Nginx). TLS termination, HTTP→HTTPS redirect, and — in the MVP single-VPS topology — the actual load-balancing/reverse-proxy layer in front of the containerized services (§19.1). At Scale, this role is taken over by a cloud load balancer in front of horizontally scaled API Gateway pods (§19.4).

API Gateway. The single authenticated entry point for all client traffic. Owns: JWT verification, RBAC policy enforcement (coarse-grained — is this role allowed to call this route at all — with fine-grained, resource-level checks pushed down to the owning service, §15.2), per-IP and per-account rate limiting (§15.5), request correlation-ID injection, and routing to the correct downstream service. In the MVP modular-monolith deployment (§4.6) the "downstream services" it routes to are in-process module calls; at Scale they become real network calls to independently deployed services, and the Gateway's routing table is the only thing that changes.

Domain Services. Described fully in §18. Each owns its own tables (schema-per-service is the *logical* boundary even when physically co-located in one Postgres instance at MVP, §6.1) and exposes its functionality only through its documented API (§16) — no service reaches into another service's tables directly, even pre-split, so that the eventual physical split is mechanical rather than a redesign.

PostgreSQL. The system of record for every durable entity in the platform: users, scenarios, sessions, synthetic telemetry, scores, audit logs. Full schema in §6.

Redis. Serves three distinct roles that are logically separate even though they share one process at MVP cost tiers (§20): (1) an application cache (entity-profile reads, leaderboard computations), (2) the pub/sub backbone for WebSocket fan-out across API Gateway replicas (§5.10), and (3) the broker for the background job queue (§5.9). At Scale, these three roles can be split onto separate Redis instances/clusters without any application code change, since the client abstraction (§18) already addresses them as three separate logical connections.

Object Storage (S3-compatible). Holds large or immutable binary/blob content: pre-generated scenario telemetry bundles (§7.7), exported PDF reports/certificates, uploaded instructor scenario assets (email templates, malware sample metadata — never real malware, §12.1), and database backup artifacts (§3.11). MinIO self-hosted at MVP; swappable for AWS S3/Cloudflare R2/Backblaze B2 at Scale via the same S3-compatible API (§5.11, §20).

Background Worker Pool. Consumes jobs from the Redis-backed queue (§5.9): telemetry generation (§7), alert correlation (§8), scoring (§12.4), email delivery, certificate rendering, scheduled analytics rollups (§5.12). Horizontally scalable independent of API replica count (§3.3).

Telemetry Generator Service, Alert Engine, Scoring Engine. These three are drawn as a pipeline because that is their logical data dependency (you cannot generate alerts without telemetry, and you cannot score an

investigation without knowing which alerts/ground-truth techniques existed) — but they are three separately deployable services/worker types, not one monolith, each detailed in §7, §8, and §12.4 respectively.

4.3 Request Lifecycle Example (End-to-End)

To make the architecture concrete, trace a single representative user action — a Student opening the Device Portal for a specific device inside an active scenario session:

6. Browser sends `GET /api/v1/sessions/{sessionId}/devices/{deviceId}` with `Authorization: Bearer <access_token>` over HTTPS.
7. Nginx terminates TLS, forwards to the API Gateway.
8. API Gateway validates the JWT signature and expiry, extracts `user_id` and `role`, checks the coarse RBAC policy for this route (Student role permitted), generates a `correlation_id`, and forwards the request to the Device Investigation Service (§10) — an in-process module call at MVP, an HTTP call to a separate service at Scale.
9. Device Investigation Service verifies **resource-level** authorization: does `sessionId` belong to `user_id` (or is the caller an Instructor/Admin with legitimate cross-session access, §15.2)? If not, 403.
10. Service checks Redis cache for this device profile (`device_profile:{sessionId}:{deviceId}`); on a cache miss, queries Postgres (`devices`, `process_events`, `network_events`, `file_events`, `registry_events` — §6 device/event tables) scoped to `sessionId`, assembles the response DTO, writes it to Redis with a short TTL (§5.6).
11. Service logs the read as an `investigation_action` of type `view_entity` (§6.13) — used later for scoring's "did the Student actually look at this evidence" signal and for the Global Timeline replay feature.
12. Response returns through the Gateway to the Browser; correlation ID is included in the response header for client-side error reporting correlation.

This same pattern (Gateway → domain service → resource-level authz → cache-or-DB → structured logging of the action) is the template for every read endpoint in §16.

4.4 Real-Time Path

WebSocket connections are established directly to the API Gateway (which proxies the upgrade to a lightweight Realtime module, §5.10) and authenticated with the same JWT used for REST calls. When the Alert Engine (background worker) emits a new alert for a live-mode scenario, it publishes to a Redis pub/sub channel scoped to `session:{sessionId}`; every API Gateway replica subscribed to that pattern forwards the event to any locally-connected WebSocket client for that session. This pub/sub indirection is what allows the Realtime module to be horizontally scaled (§3.3) — a worker never needs to know which specific Gateway replica holds a given Student's socket.

4.5 Trust Boundaries

Three trust boundaries are enforced end-to-end and referenced throughout §15:

13. **Public Internet ↔ Load Balancer.** Only TLS 443 (and 80 for redirect) is exposed. No other port is internet-reachable in any deployment topology.
14. **Load Balancer/Gateway ↔ Internal Services.** Internal service-to-service traffic (at Scale, once physically split) occurs on a private network/VPC with no public ingress; MVP's modular monolith has no network hop here at all, which is strictly more secure by default and is one of the deliberate advantages of the phased topology (§4.6).
15. **Services ↔ Data Stores.** Postgres, Redis, and object storage are never directly reachable from the public internet in any phase; credentials are injected via the secrets mechanism in §5.16, never hardcoded.

4.6 Why a "Modular Monolith" at MVP

The architecture in §4.1 is drawn as if services are already physically separate because that is the *contract* every module must honor from day one (§3.7). But the actual MVP deployment (§19.1) runs the API Gateway and all domain services as **one process** (a modular monolith) for three concrete reasons:

- **Cost.** One compute process is dramatically cheaper to host than five-plus independently deployed services with their own resource reservations, at a traffic level (§3.3, 500 concurrent sessions) that doesn't need the isolation.
- **Latency.** In-process module calls avoid network hops entirely, which materially helps hit the aggressive MVP latency targets in §3.1 on modest hardware.
- **Operational simplicity for a small team.** One deployable artifact means one CI/CD pipeline, one set of logs to tail, one process to restart — appropriate for the team size implied by the cost constraints in §20.

The module boundaries (§18) are still enforced at the *code* level (each domain service is its own package/namespace with an explicit public interface and no direct cross-module table access) specifically so that the Scale-phase migration (§19.4) to physically separate, independently scaled services is a deployment and dependency-injection change, not a rewrite of business logic.

5. Infrastructure Design

5.1 Technology Stack Summary

Layer	MVP Choice	Rationale
Frontend	React + TypeScript, Vite build	Large talent pool, mature ecosystem, fast dev iteration (§17)
API Gateway / Backend	Node.js (TypeScript) on NestJS, OR equivalent typed backend framework	Single language across FE/BE reduces team size needed (§20); NestJS's module system maps 1:1 onto the domain-service boundaries in §18
Database	PostgreSQL 15+	Relational integrity for grading-critical data, JSONB for flexible telemetry payloads, mature partitioning/replication story (§6)
Cache / Queue Broker / Pub-Sub	Redis 7+	One dependency serving three roles at MVP cost (§4.2), trivially split later
Object Storage	MinIO (self-hosted, S3 API-compatible)	Free, self-hosted, drop-in swap for AWS S3/R2/B2 later (§5.11)
Reverse Proxy / LB	Nginx	Free, battle-tested, doubles as static file server for MVP
Containerization	Docker + Docker Compose (MVP) → Kubernetes (Scale)	§19
Background Jobs	BullMQ (Redis-backed)	Mature Node.js job queue with retries, scheduling, dashboards
CI/CD	GitHub Actions	Free tier sufficient at MVP scale, industry standard

This stack is a recommendation consistent with the cost and portability constraints in §3.6/§20, not a hard architectural requirement — any typed backend framework and any Postgres/Redis/S3-compatible trio satisfies the architecture in §4. What **is** required is that every choice remain swappable per the abstractions in §5.6–§5.11.

5.2 API Layer

The API Gateway is the single ingress for REST and WebSocket traffic (§4.2). It owns:

- **Routing** to domain-service modules (in-process at MVP, service-mesh/HTTP at Scale) via a declarative route table (verb + path → handler), so the routing table itself is the artifact that changes during the Scale migration, not the handlers.
- **AuthN**: JWT verification (§5.7).
- **Coarse AuthZ**: role-route matrix (§15.2) — is this role ever allowed to hit this route.

- **Correlation ID** generation/propagation (**X-Correlation-Id** header), attached to every downstream log line and error response for support traceability.
- **Rate limiting** (§15.5), implemented as Redis-backed sliding-window counters keyed by **user_id** (authenticated) or IP (unauthenticated).
- **Request validation** at the edge (schema validation against the OpenAPI contract, §16) before any handler code runs, rejecting malformed requests with 400 before they reach business logic.

All REST endpoints are versioned under **/api/v1/...** from day one (§16.1) so a breaking v2 can be introduced without disrupting existing frontend/mobile clients.

5.3 Database Layer

Single PostgreSQL instance at MVP (§19.1), with logical schema separation by domain service (**identity**, **investigation**, **scenario**, **learning**, **audit** — mirroring §18) even though physically one database, enforced by convention + migration tooling (§6.24) rather than physical instance separation, to keep MVP operational overhead low while preserving a clean seam for later physical separation if a specific service's load ever requires its own database (§19.4).

Connection pooling via PgBouncer (or the ORM's built-in pool at MVP scale) sits between the application and Postgres to avoid connection exhaustion under the concurrent-session targets in §3.3.

5.4 Caching

Redis serves as the application cache for: entity-profile reads (**identity/device/mailbox** detail views, §9–§11), session metadata, computed leaderboard rankings (§13.5), and rate-limit counters (§15.5). Cache keys are namespaced **{domain}:{sessionId}:{entityId}** with short TTLs (30–120s) for investigation data (freshness matters more than cache hit rate here, since scoring correctness depends on the Student seeing current data) and longer TTLs (hours) for mostly-static content like published scenario metadata and MITRE technique reference data.

Cache invalidation strategy: **write-through** for anything the platform itself mutates (evidence pin, note save — invalidate the specific key on write) and **TTL-expiry only** for data that is generated once and never mutated post-generation (telemetry, alerts) — the latter case needs no invalidation logic at all, which is a deliberate simplification enabled by the "generate once, then read-only" design of scenario telemetry (§7.7).

5.5 Authentication & Authorization (Infrastructure View)

Full policy detail in §15. Infrastructure summary: credentials stored as argon2id hashes; short-lived (15 min) signed JWT access tokens plus longer-lived (30 day), rotating, revocable refresh tokens stored hashed in Postgres (**refresh_tokens** table, §6.19); SSO (SAML 2.0 / OIDC) supported at the Org Admin tier for institutional buyers (§2.16) as a Phase 2 addition, implemented as an alternate token-issuance path feeding the same internal JWT format so no downstream service needs to know whether a session originated from password auth or SSO.

5.6 Data Access Abstraction

Every domain service accesses Postgres exclusively through a typed repository layer (no raw SQL scattered through business logic, no ORM query building outside the repository layer) and accesses Redis exclusively through a small internal `CacheClient/QueueClient/PubSubClient` interface (§4.2). This is the concrete mechanism that makes "swap Redis roles onto separate instances later" (§4.2) and "swap MinIO for S3" (§5.11) zero-application-code-change operations: only the client implementation's connection config changes.

5.7 Authentication Token Details

- Access token: JWT, HS256 at MVP (single-service secret, §5.16) or RS256 at Scale (public-key verification without a network call to the auth service from every replica), 15-minute expiry, claims: `sub` (user id), `role`, `org_id` (nullable), `session_version` (incremented on password change/logout-all to allow instant invalidation of already-issued access tokens without a blocklist, §15.7).
- Refresh token: opaque random 256-bit token, stored **hashed** (not plaintext) in `refresh_tokens`, rotated on every use (old token invalidated, new one issued — detects token replay/theft), 30-day sliding expiry, revocable individually (single device logout) or in bulk (logout-all-devices, security-incident response).

5.8 Containerization

Every service (API Gateway/monolith at MVP, each split service at Scale), the worker pool, Postgres, Redis, MinIO, and Nginx run as Docker containers. A single `docker-compose.yml` at MVP (§19.1) defines the full stack for both local development and production VPS deployment, with environment-specific override files (`docker-compose.prod.yml`) rather than divergent configs, so "works in dev" and "works in prod" are the same container images with different env vars (§5.17) — eliminating an entire class of environment-drift bugs.

5.9 Message Queues & Background Workers

BullMQ (Redis-backed) provides named queues, each with its own worker pool and concurrency limit, isolating slow/bursty job types from each other:

Queue	Producer	Consumer	Notes
<code>telemetry-generation</code>	Scenario publish action / session start	Telemetry Generator worker (§7)	Heaviest CPU job; concurrency capped per worker to protect the shared VPS at MVP
<code>alert-correlation</code>	Telemetry Generator completion	Alert Engine worker (§8)	Chained after telemetry job via BullMQ flow/dependency feature
<code>scoring</code>	Scenario submission	Scoring Engine worker (§12.4)	Must complete fast (§3.1) since the Student is watching a results screen
<code>email</code>	Various (welcome, cert issued, cohort invite)	Email worker	Uses a transactional email provider's API, not self-

Queue	Producer	Consumer	Notes
			hosted SMTP (§20 — cheap, avoids deliverability pain)
reporting	PDF export request, scheduled analytics rollup	Reporting worker	Lower priority, can tolerate multi-second latency

All jobs are **idempotent** by design (§3.8): each job payload carries a natural idempotency key (e.g., `sessionId` for telemetry generation), and handlers check-then-act against Postgres state so a duplicate delivery (BullMQ's at-least-once guarantee) never double-generates telemetry or double-applies a score.

5.10 WebSockets

A lightweight Realtime module inside the API Gateway (§4.4) manages WebSocket upgrade, auth (JWT passed as a connection query param or first-message auth frame), and per-session channel subscription. Server→client message types are a small, versioned, documented set (`alert.new`, `alert.updated`, `incident.status_changed`, `timeline.updated`) — the WebSocket channel is a push-notification convenience, never the sole source of truth for any data (§3.8's reliability principle extends here: the frontend must always be able to reconstruct current state via REST GET if a socket message is missed, §17.9).

5.11 Object Storage

S3-compatible API (MinIO self-hosted at MVP, §19.1) with buckets: `scenario-assets` (instructor-uploaded email templates, images, decoy documents used in scenario authoring, §12.1), `pregenerated-telemetry` (§7.7's cached telemetry bundles), `exports` (PDF incident reports, gradebook CSVs), `certificates` (issued certificate PDFs), and `backups` (§3.11, replicated to a separate provider/region). All buckets are private by default; client access to any object is via short-lived signed URLs issued by the owning service, never public bucket ACLs (§15.6).

5.12 Scheduled Jobs

A cron-style scheduler (BullMQ's repeatable jobs at MVP; a Kubernetes CronJob at Scale, §19.4) drives: nightly database backup (§3.11), nightly analytics rollup (pre-aggregating §2.17's dashboards so they don't compute from raw tables on every admin page load), weekly stale-session cleanup (sessions abandoned mid-scenario past a TTL are archived, freeing their telemetry from hot storage per §6.23's retention policy), and monthly leaderboard-period rollover (§13.5).

5.13 Deployment Architecture

Detailed in §19. Summary: MVP is Docker Compose on a single VPS behind Nginx with Let's Encrypt-issued TLS certs, auto-renewed via certbot's standard cron mechanism. Scale phase is a managed Kubernetes cluster with the same container images, a managed Postgres (RDS/Cloud SQL-equivalent) replacing self-hosted Postgres, and a managed object storage service replacing self-hosted MinIO — a lift, not a rewrite, because of the abstraction discipline in §5.6.

5.14 CDN

Static frontend assets and public pages (§4.2) are served through a CDN from day one — this is one of the few "spend a little even at MVP" infrastructure choices, because CDN tiers suitable for this traffic level are free or near-free (Cloudflare's free tier, or the CDN bundled with the chosen static-hosting provider) and the latency/cost benefit (§3.1, §20) is disproportionate to the near-zero cost.

5.15 Load Balancer

MVP: Nginx performs both TLS termination and (trivial, single-upstream) load balancing to the one application container. Scale: a cloud provider's managed load balancer (or an Nginx/HAProxy tier run as its own scaled deployment) distributes traffic across N API Gateway replicas using least-connections or round-robin, with active health checks against each replica's `/health` endpoint (§3.7) removing unhealthy replicas from rotation automatically.

5.16 Monitoring & Logging (Infrastructure View)

Prometheus + Grafana (self-hosted containers at MVP, §19.1) scrape `/metrics` from every service and worker for the golden signals in §3.10. Loki (self-hosted, same Grafana stack) aggregates structured JSON logs (§3.9) from all containers via Docker's logging driver. At Scale, these are either kept self-hosted (now justified by higher traffic making self-hosting's fixed cost worthwhile relative to a managed observability vendor) or swapped for a managed observability provider — a deployment-config decision made on cost data at the time, not an architectural one.

5.17 Secrets Management

MVP: secrets (database credentials, JWT signing key, object storage keys, transactional email API key) are injected as environment variables from a `.env` file that is **never committed to source control** (enforced via `.gitignore` and a pre-commit secret-scanning hook, §15.11), loaded by Docker Compose's `env_file` directive. Scale: secrets move to a dedicated secrets manager (cloud provider's native secrets manager, or a self-hosted Vault) injected into Kubernetes pods via mounted secret volumes or the CSI secrets-store driver, removing plaintext `.env` files from any host entirely. In both phases, secrets are rotated on any suspected exposure and on a routine schedule for the JWT signing key specifically (§15.7).

5.18 Environment Variables

Every service reads configuration exclusively from environment variables validated at process startup against a strict schema (fail fast on missing/malformed config rather than failing later on first use) — covering database/Redis/object-storage connection strings, JWT secret/algorithm, token TTLs, external email provider API key, feature flags (§2.16), and the deployment environment name (`development` / `staging` / `production`) used only for logging/observability tagging, never for behavioral branching (§3.7's "no environment-specific code branches" principle).

6. Database Design

6.1 Conventions & Tenancy Model

- **Primary keys:** every table uses a `UUID (v4)` primary key named `id`, generated application-side or via Postgres `gen_random_uuid()`. UUIDs are used instead of serial integers so that IDs are safe to expose in URLs/APIs without leaking row-count/growth-rate information, and so that pre-generated telemetry (§7.7) can assign IDs before insertion without a round-trip.
- **Timestamps:** every table has `created_at TIMESTAMPTZ NOT NULL DEFAULT now()`; mutable tables also have `updated_at TIMESTAMPTZ NOT NULL DEFAULT now()` maintained by a trigger. All timestamps are stored UTC; timezone conversion is a presentation-layer concern only (§17).
- **Soft delete vs. hard delete:** user-owned content that a Student might reasonably want "undone" (notes, evidence pins) is hard-deleted on explicit removal — there is no undo-from-trash feature at MVP. Records with compliance/audit weight (users, audit_logs, scores, certificates) are never hard-deleted; deactivation uses a `deleted_at TIMESTAMPTZ NULL` (soft delete) column instead, and all queries against these tables filter `WHERE deleted_at IS NULL` by default via the repository layer (§5.6).
- **Tenancy model:** SOCVerse is a single database, shared-schema multi-tenant system. Every row that belongs to an institutional customer carries an `org_id UUID NULL REFERENCES organizations(id)` (nullable because individual B2C users have no org). Row-level isolation between orgs is enforced at the application/repository layer (every query scopes by the caller's `org_id` where applicable) rather than Postgres Row-Level Security at MVP, to keep the query layer simple; **migrating to native Postgres RLS is the specified hardening step before onboarding the first enterprise customer with a compliance requirement for defense-in-depth tenant isolation** (§15, noted as a Scale-phase action item, not required for MVP launch).
- **Naming:** tables are `snake_case`, plural nouns. Foreign keys are named `{referenced_table_singular}_id`. Enum-like fields use Postgres `CHECK` constraints or native `ENUM` types (native enums preferred for fields with a small, rarely-changing value set like `severity`; `CHECK` constraints preferred where the value set may grow, to avoid enum-alteration migration friction).

6.2 Entity-Relationship Overview

```

organizations 1—* cohorts 1—* cohort_enrollments *—1 users
users 1—* investigation_sessions *—1 attack_scenarios
attack_scenarios 1—* scenario_techniques *—1 mitre_techniques
investigation_sessions 1—* identities, devices, email_messages (session-scoped synthetic
entities)
identities 1—* sign_in_events
devices 1—* process_events, file_events, network_events, dns_events, registry_events
investigation_sessions 1—* alerts 1—* alert_evidence_refs
investigation_sessions 1—* incidents *—* alerts (via incident_alerts)
incidents 1—* evidence_collection, analyst_notes, investigation_actions
investigation_sessions 1—1 scores
incidents 1—* instructor_feedback
users 1—* certificates
users 1—* refresh_tokens

```

(all mutating actions) —> audit_logs

Every synthetic entity (identity, device, email message, and every event table) carries a `session_id` UUID NOT NULL REFERENCES `investigation_sessions(id)` ON DELETE CASCADE, because synthetic telemetry has no meaning outside the session that generated it (§7) — this is the single most important foreign key in the schema, and it is what makes per-session data isolation and eventual archival/partitioning (§6.23) straightforward.

6.3 `users` and `roles`

`users`

Column	Type	Constraints	Notes
id	UUID	PK	
org_id	UUID	FK → organizations, NULL	NULL for individual B2C accounts
email	CITEXT	UNIQUE NOT NULL	case-insensitive
password_hash	TEXT	NULL	NULL if SSO-only account; argon2id, §15.1
role	ENUM	NOT NULL	student, instructor, org_admin, platform_admin (§15.2)
display_name	TEXT	NOT NULL	
status	ENUM	NOT NULL DEFAULT 'active'	active, suspended, pending_verification
email_verified_at	TIMESTAMPTZ	NULL	
sso_provider	TEXT	NULL	saml, oidc, NULL if password auth
sso_subject	TEXT	NULL	provider-side subject identifier
session_version	INTEGER	NOT NULL DEFAULT 1	incremented to invalidate all issued access tokens (§5.7)
last_login_at	TIMESTAMPTZ	NULL	
deleted_at	TIMESTAMPTZ	NULL	soft delete (§6.1)
created_at / updated_at	TIMESTAMPTZ	NOT NULL	

Indexes: `UNIQUE (email) WHERE deleted_at IS NULL`; `INDEX (org_id)`.

A separate `roles` table is **not** used as a many-to-many join for MVP — the four platform roles in §15.2 are coarse and mutually exclusive per user, so a single `role` enum column is the correct normalization (avoiding

an unnecessary join on every authorization check, which is a hot path). Fine-grained **permissions** (e.g., "can this specific instructor edit this specific cohort") are modeled as ownership foreign keys (`cohorts.owner_id`) checked at the resource-level authorization step (§4.3), not as a separate permissions table. If SOCVerse later needs custom/composable roles (e.g., enterprise customers wanting a "read-only auditor" role), a `roles` and `user_roles` join table would be introduced then; the `role` enum column is kept as a computed convenience field for backward compatibility rather than removed, to avoid a breaking change to every existing authorization check.

6.4 `organizations`

Column	Type	Constraints	Notes
id	UUID	PK	
name	TEXT	NOT NULL	
plan	ENUM	NOT NULL	trial, institutional, enterprise (§1.7)
seat_limit	INTEGER	NOT NULL	
sso_enabled	BOOLEAN	NOT NULL DEFAULT false	
sso_config	JSONB	NULL	provider metadata (§5.5), secrets referenced not embedded (§15.8)
billing_customer_id	TEXT	NULL	external billing provider reference, §20
created_at / updated_at	TIMESTAMPTZ	NOT NULL	

6.5 `cohorts`

Column	Type	Constraints	Notes
id	UUID	PK	
org_id	UUID	FK → organizations NOT NULL	
owner_id	UUID	FK → users NOT NULL	the Instructor who owns this cohort
name	TEXT	NOT NULL	
join_code	TEXT	UNIQUE NULL	self-service enrollment (§2.15)
starts_at / ends_at	DATE	NULL	
created_at / updated_at	TIMESTAMPTZ	NOT NULL	

6.6 `cohort\_enrollments`

Column	Type	Constraints	Notes
id	UUID	PK	
cohort_id	UUID	FK → cohorts NOT NULL	
user_id	UUID	FK → users NOT NULL	
enrolled_at	TIMESTAMPTZ	NOT NULL	
status	ENUM	NOT NULL DEFAULT 'active'	active, dropped

Indexes: `UNIQUE (cohort_id, user_id)`.

Also `cohort_scenario_assignments` (id, cohort_id FK, scenario_id FK, due_at, attempt_limit, created_by FK→users) supports §2.15's assignment feature.

6.7 `attack\_scenarios`

Column	Type	Constraints	Notes
id	UUID	PK	
slug	TEXT	UNIQUE NOT NULL	URL-safe identifier
title	TEXT	NOT NULL	
summary	TEXT	NOT NULL	learner-facing teaser, does not leak ground truth
category	ENUM	NOT NULL	identity, endpoint, email, cloud, insider_threat, web, malware, ransomware (§1.8)
difficulty	ENUM	NOT NULL	beginner, intermediate, advanced, expert (§12.2)
estimated_minutes	INTEGER	NOT NULL	
status	ENUM	NOT NULL DEFAULT 'draft'	draft, published, archived (§12.6)
current_version_id	UUID	FK → scenario_versions NULL	points at the live version (§12.6)
author_id	UUID	FK → users NOT NULL	
created_at / updated_at	TIMESTAMPTZ	NOT NULL	

``scenario_versions`` (append-only, §12.6): `id`, `scenario_id` FK, `version_number` INTEGER, `ground_truth_definition` JSONB (the authored attack narrative: entities, event templates, injected IOCs, correct technique tags, rubric weights — the schema of this JSON blob is defined in §12.1), `published_at`, `created_by` FK→users. `attack_scenarios.current_version_id` is only repointed after a new version passes the scenario validation pipeline (§12.6), so in-flight sessions always continue against the version they started on (`investigation_sessions.scenario_version_id`, §6.9) even if the scenario is edited mid-session.

``scenario_techniques`` (join): `scenario_version_id` FK, `mitre_technique_id` FK, `is_required_for_full_credit` BOOLEAN — drives the technique-identification scoring component (§12.4).

6.8 ``mitre_techniques``

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK	
<code>technique_id</code>	TEXT	UNIQUE NOT NULL	e.g. T1078, official ATT&CK ID
<code>name</code>	TEXT	NOT NULL	e.g. "Valid Accounts"
<code>tactic</code>	TEXT	NOT NULL	e.g. TA0001 Initial Access — a technique can map to multiple tactics; modeled via a <code>technique_tactics</code> join if strict correctness is required, or a denormalized primary-tactic column if single-tactic display is sufficient for MVP UI (MVP choice: denormalized column + optional <code>secondary_tactics</code> TEXT[])
<code>description</code>	TEXT	NOT NULL	
<code>url</code>	TEXT	NULL	reference link to attack.mitre.org for learner further-reading, not a runtime dependency

Seeded once from the public MITRE ATT&CK STIX dataset at deploy time (an ingestion script, not a live API dependency — §1.6's "no live external dependency" principle applies here too), refreshed periodically as ATT&CK is updated.

6.9 ``investigation_sessions``

The central join point between a user, a scenario, and the synthetic world generated for that attempt.

Column	Type	Constraints	Notes
id	UUID	PK	this is the <code>session_id</code> / correlation scope referenced by every synthetic table below and by §2.9's cross-entity correlation
user_id	UUID	FK → users NOT NULL	
scenario_id	UUID	FK → attack_scenarios NOT NULL	
scenario_version_id	UUID	FK → scenario_versions NOT NULL	pinned at session start (§6.7)
cohort_assignment_id	UUID	FK → cohort_scenario_assignments NULL	NULL for self-directed practice
status	ENUM	NOT NULL DEFAULT 'active'	active, submitted, scored, abandoned
seed	BIGINT	NOT NULL	RNG seed used by the Telemetry Generator (§7.2) — stored for reproducibility/debugging, never exposed to the client
started_at	TIMESTAMPTZ	NOT NULL	
submitted_at	TIMESTAMPTZ	NULL	
expires_at	TIMESTAMPTZ	NOT NULL	stale-session cleanup horizon (§5.12)
current_scenario_time	TIMESTAMPTZ	NULL	for live/time-phased scenarios (§12.7), the simulated "now"

Indexes: INDEX (user_id, status), INDEX (scenario_id), INDEX (expires_at) WHERE status = 'active' (used by the cleanup job, §5.12).

6.10 `identities`

Synthetic directory entries for the Identity Portal (§9).

Column	Type	Constraints	Notes
id	UUID	PK	

Column	Type	Constraints	Notes
session_id	UUID	FK → investigation_sessions ON DELETE CASCADE NOT NULL	
display_name	TEXT	NOT NULL	synthetic person name
user_principal_name	TEXT	NOT NULL	synthetic user@scenario-corp.example
department	TEXT	NOT NULL	
job_title	TEXT	NOT NULL	
manager_identity_id	UUID	FK → identities NULL	self-referential
risk_level	ENUM	NOT NULL	none, low, medium, high — computed by the Alert Engine (§8), not authored directly
mfa_status	ENUM	NOT NULL	enforced, registered_not_enforced, not_registered
account_status	ENUM	NOT NULL	active, disabled, locked — mutable via the in-scenario "Disable Account" investigation action (§2.20)
is_privileged	BOOLEAN	NOT NULL DEFAULT false	e.g. Global Admin-equivalent — used by scenario ground truth for privilege-escalation narratives
home_country	TEXT	NOT NULL	ISO 3166-1 alpha-2, used to compute impossible-travel baselines (§9.7)
is_ground_truth_actor	BOOLEAN	NOT NULL DEFAULT false	internal-only flag marking whether this identity is part of the authored attack narrative; never returned by any Student-facing API (§9.9)

``identity_group_memberships``: identity_id FK, group_name TEXT, group_type ENUM (security, distribution, role_assignable).

``conditional_access_evaluations``: id, identity_id FK, sign_in_event_id FK → sign_in_events, policy_name, result ENUM (granted, blocked, mfa_required), reasons JSONB — supports §9's conditional access investigation surface.

6.11 `devices`

Synthetic endpoints for the Device Portal (§10).

Column	Type	Constraints	Notes
id	UUID	PK	
session_id	UUID	FK → investigation_sessions ON DELETE CASCADE NOT NULL	
hostname	TEXT	NOT NULL	
os_platform	ENUM	NOT NULL	windows, linux, macos
os_version	TEXT	NOT NULL	
primary_identity_id	UUID	FK → identities NULL	primary logged-on user
risk_level	ENUM	NOT NULL	computed, mirrors identities.risk_level semantics
isolation_status	ENUM	NOT NULL DEFAULT 'not_isolated'	not_isolated, isolated — mutable via §2.20's containment action
last_seen_at	TIMESTAMPTZ	NOT NULL	
is_ground_truth_actor	BOOLEAN	NOT NULL DEFAULT false	internal-only, mirrors §6.10

`installed\_software`: device_id FK, name, version, install_date, is_known_bad BOOLEAN (internal-only).

`device\_services`: device_id FK, service_name, start_type, status, binary_path — persistence-artifact investigation.

`startup\_entries`: device_id FK, entry_type ENUM(run_key, scheduled_task, startup_folder, service, wmi_subscription), command, location.

`usb\_events`: device_id FK, event_type ENUM(connected, disconnected), device_serial, volume_label, occurred_at.

6.12 Telemetry Event Tables

All event tables below share a common shape and are the direct implementation of §7's "realistic synthetic telemetry" requirement and §2.8's Search feature. Every event table has: id UUID PK, session_id UUID FK NOT NULL, occurred_at TIMESTAMPTZ NOT NULL, correlation_id UUID NULL (links events that are part of the same real-world action across entities, e.g., a sign-in and the subsequent process launch it caused — used by §2.9's cross-entity timeline correlation lines), raw JSONB NOT NULL (the full synthetic event payload in a shape resembling the real-world log source, so the Search UI's freetext search and the "raw event" detail view have realistic depth to explore), is_ground_truth_evidence BOOLEAN NOT NULL DEFAULT false (internal-only: was this event authored as part of the attack narrative, vs.

background noise — the single most important column for scoring, §12.4), and `mitre_technique_id` UUID FK NULL (internal-only, set only on ground-truth evidence rows).

6.12.1 `sign\_in\_events`

`identity_id` FK, `device_id` FK NULL, `source_ip` INET, `source_country` TEXT, `source_city` TEXT, `application`, `result` ENUM(success,failure,mfa_denied,blocked_by_ca), `failure_reason` TEXT NULL, `is_legacy_auth` BOOLEAN, `client_app` TEXT. Indexed on (`session_id`, `identity_id`, `occurred_at`) — the primary access pattern for the Identity Portal's sign-in timeline (§9).

6.12.2 `process\_events`

`device_id` FK, `process_guid` UUID, `parent_process_guid` UUID NULL, `image_path` TEXT, `command_line` TEXT, `hash_sha256` TEXT, `parent_image_path` TEXT NULL, `integrity_level` TEXT, `identity_id` FK NULL (the logged-on user context). This is the Sysmon-Event-ID-1-shaped table backing the Device Portal's process tree (§10.3); `parent_process_guid` self-references within the same `device_id` to reconstruct the tree.

6.12.3 `file\_events`

`device_id` FK, `action` ENUM(created,modified,deleted,renamed,encrypted), `file_path`, `hash_sha256` NULL, `process_guid` FK NULL (which process performed the action) — `encrypted` action type specifically supports ransomware scenarios (§1.8).

6.12.4 `network\_events`

`device_id` FK, `direction` ENUM(inbound,outbound), `protocol`, `local_port`, `remote_ip` INET, `remote_port`, `bytes_sent`, `bytes_received`, `process_guid` FK NULL — supports C2 beaconing and exfiltration-volume investigation.

6.12.5 `dns\_events`

`device_id` FK, `query_name`, `query_type`, `response_ips` INET[], `process_guid` FK NULL — supports C2 domain and DGA-style investigation narratives.

6.12.6 `http\_requests`

`device_id` FK NULL, `identity_id` FK NULL, `method`, `url`, `user_agent`, `status_code`, `request_headers` JSONB, `source_ip` INET — supports web attack scenarios (webshell access patterns, injection attempts) and proxy-log-style investigation.

6.12.7 `registry\_events`

`device_id` FK, `action` ENUM(created,modified,deleted), `key_path`, `value_name` NULL, `value_data` NULL, `process_guid` FK NULL — Windows persistence-artifact investigation (Run keys, etc.), feeding `startup_entries` (§6.11) reconciliation.

6.12.8 `firewall\_events`, `vpn\_events`, `proxy\_events`, `cloud\_events`

Same base shape; `firewall_events` (action ENUM allow/deny, source/dest IP+port, rule_name), `vpn_events` (identity_id FK, source_ip, session_duration, client), `proxy_events` (mirrors `http_requests` with category classification e.g. malware, phishing, uncategorized), `cloud_events` (identity_id FK, provider ENUM(aws-style, azure-style, saas), action_name e.g. `CreateAccessKey`, resource_arn, source_ip) — collectively cover §7's Firewall/VPN/Proxy/Cloud log requirement and §1.8's Cloud investigation category.

6.12.9 `email\_messages`

Detailed further in §11. `session_id` FK, `message_id` TEXT, `direction` ENUM(inbound, outbound, internal), `sender_address`, `sender_display_name`, `recipient_addresses` TEXT[], `subject`, `body_html`, `headers_raw` JSONB (full synthetic header block), `spf_result` ENUM(pass, fail, softfail, none), `dkim_result` ENUM(pass, fail, none), `dmARC_result` ENUM(pass, fail, none), `is_ground_truth_evidence` BOOLEAN, `mitre_technique_id` FK NULL.

`email\_attachments`: `email_message_id` FK, `filename`, `content_type`, `size_bytes`, `hash_sha256`, `sandbox_verdict` ENUM(benign, suspicious, malicious, not_analyzed).

`email\_urls`: `email_message_id` FK, `url`, `display_text`, `reputation` ENUM (mirrors §2.7's TI reputation scale), `is_rewritten_by_safe_links` BOOLEAN.

6.13 `investigation\_actions`

The audit/replay trail of everything a Student *does* during a session (distinct from `audit_logs`, §6.22, which covers platform-security-relevant actions across the whole product, not just in-scenario investigative actions).

Column	Type	Constraints	Notes
id	UUID	PK	
session_id	UUID	FK → investigation_sessions NOT NULL	
incident_id	UUID	FK → incidents NULL	
user_id	UUID	FK → users NOT NULL	
action_type	ENUM	NOT NULL	view_entity, search, add_to_timeline, pin_evidence, isolate_device, disable_account, block_sender, dismiss_alert, escalate_to_incident, submit_verdict (§2.20)

Column	Type	Constraints	Notes
target_type	TEXT	NOT NULL	e.g. device, identity, email_message, alert
target_id	UUID	NOT NULL	polymorphic reference, not FK-constrained (spans many tables)
metadata	JSONB	NULL	e.g. dismissal reason, search query text
occurred_at	TIMESTAMPTZ	NOT NULL	

Indexed on (session_id, occurred_at) for the Global Timeline replay feature (§2.9) and on (session_id, action_type) for scoring queries (§12.4, e.g. "did the Student view the ground-truth-evidence entities at all").

6.14 `scores`

Column	Type	Constraints	Notes
id	UUID	PK	
session_id	UUID	FK → investigation_sessions UNIQUE NOT NULL	one score per session
overall_percent	NUMERIC(5,2)	NOT NULL	
technique_accuracy_percent	NUMERIC(5,2)	NOT NULL	correct technique tags identified / required techniques (§12.4)
evidence_precision_percent	NUMERIC(5,2)	NOT NULL	pinned evidence that was actually ground-truth / total pinned
evidence_recall_percent	NUMERIC(5,2)	NOT NULL	ground-truth evidence pinned / total ground-truth evidence
false_positive_count	INTEGER	NOT NULL	non-ground-truth items pinned as evidence or wrongly escalated
hint_penalty_percent	NUMERIC(5,2)	NOT NULL	§12.5
time_to_resolution_seconds	INTEGER	NOT NULL	
verdict_correct	BOOLEAN	NOT NULL	did the final True/False/Benign Positive verdict match ground truth

Column	Type	Constraints	Notes
rubric_breakdown	JSONB	NOT NULL	full itemized rubric result, rendered on the results screen (§2.12)
scored_at	TIMESTAMPTZ	NOT NULL	

`score\_history` (append-only, not shown in full): retains prior scoring runs if a session is re-scored after a scoring-engine bugfix, so historical leaderboard/analytics data can be reconciled rather than silently changing underfoot.

6.15 `alerts`

Column	Type	Constraints	Notes
id	UUID	PK	
session_id	UUID	FK → investigation_sessions NOT NULL	
title	TEXT	NOT NULL	
description	TEXT	NOT NULL	
severity	ENUM	NOT NULL	informational, low, medium, high, critical (§8.5)
status	ENUM	NOT NULL DEFAULT 'new'	new, in_progress, resolved, dismissed
dismissal_reason	TEXT	NULL	required when status → dismissed (§2.1)
primary_entity_type	ENUM	NOT NULL	identity, device, mailbox
primary_entity_id	UUID	NOT NULL	polymorphic
mitre_technique_id	UUID	FK → mitre_techniques NULL	
detection_rule_id	UUID	FK → detection_rules NULL	which rule fired (§8.1)
dedup_count	INTEGER	NOT NULL DEFAULT 1	§8.7
is_false_positive_by_design	BOOLEAN	NOT NULL DEFAULT false	internal-only, marks intentionally-noisy decoy alerts (§8.6)
first_seen_at / last_seen_at	TIMESTAMPTZ	NOT NULL	

`alert\_evidence\_refs`: `alert_id` FK, `event_table` TEXT (which of the §6.12 tables), `event_id` UUID (polymorphic reference to the specific row) — this is what makes an alert's "show evidence" drill-down (§2.1) deterministic rather than a re-search.

`detection\_rules`: `id`, `name`, `description`, `logic_summary` TEXT (human-readable, not an executable query language — §8.1), `default_severity`, `mitre_technique_id` FK, `is_active` BOOLEAN.

`threat\_intel\_indicators` (§2.7): `id`, `scenario_version_id` FK, `indicator_type` ENUM(hash,ip,domain,url), `value`, `reputation` ENUM, `actor_attribution` TEXT NULL, `context` TEXT.

6.16 `incidents`

Column	Type	Constraints	Notes
<code>id</code>	UUID	PK	
<code>session_id</code>	UUID	FK → investigation_sessions NOT NULL	
<code>title</code>	TEXT	NOT NULL	Student-authored
<code>status</code>	ENUM	NOT NULL DEFAULT 'open'	open, investigating, contained, closed, reopened (§2.3)
<code>verdict</code>	ENUM	NULL	true_positive, false_positive, benign_positive — set at closure
<code>summary</code>	TEXT	NULL	required at closure
<code>closed_at</code>	TIMESTAMPTZ	NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMPTZ	NOT NULL	

`incident\_alerts` (join): `incident_id` FK, `alert_id` FK, `linked_at` — supports §2.2's "build one incident from multiple alerts."

`incident\_techniques` (join): `incident_id` FK, `mitre_technique_id` FK — the Student's own tagging, compared against `scenario_techniques` (§6.7) at scoring time.

6.17 `evidence\_collection` and `analyst\_notes`

`evidence\_collection`: `id`, `incident_id` FK NOT NULL, `event_table` TEXT, `event_id` UUID (polymorphic, mirrors `alert_evidence_refs`), `justification` TEXT NOT NULL, `mitre_technique_id` FK NULL (Student's own tag), `pinned_by` FK→users, `pinned_at`.

`analyst\_notes`: `id`, `incident_id` FK NOT NULL, `body` TEXT NOT NULL (constrained rich text, §2.13), `created_by` FK→users, `created_at`, `updated_at`.

6.18 `certificates`

Column	Type	Constraints	Notes
id	UUID	PK	also serves as the public verification token (§2.18)
user_id	UUID	FK → users NOT NULL	
learning_path_id	UUID	FK → learning_paths NOT NULL	
issued_at	TIMESTAMPTZ	NOT NULL	
score_snapshot	JSONB	NOT NULL	denormalized summary of qualifying scores, so the certificate remains valid/renderable even if underlying session data is later archived per §6.23
pdf_object_key	TEXT	NOT NULL	object storage reference (§5.11)
revoked_at	TIMESTAMPTZ	NULL	integrity mechanism if a certificate is later found to be issued in error

Also, from the Learning Platform domain (§13): `courses`, `learning_paths`, `learning_path_scenarios` (join, ordered), `achievements`, `user_achievements`, `leaderboard_entries` (`user_id`, `period` ENUM(`weekly`,`monthly`,`all_time`), `points`, `rank` — recomputed by the scheduled rollout job, §5.12).

6.19 `refresh\_tokens`

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	FK → users NOT NULL	
token_hash	TEXT	UNIQUE NOT NULL	SHA-256 of the opaque token; plaintext never stored (§5.7)
device_label	TEXT	NULL	user-facing "which device" for the logout-one-device UI
issued_at	TIMESTAMPTZ	NOT NULL	
expires_at	TIMESTAMPTZ	NOT NULL	
revoked_at	TIMESTAMPTZ	NULL	set on rotation-detected-reuse (theft signal), explicit logout, or bulk revoke

Column	Type	Constraints	Notes
replaced_by_token_id	UUID	FK → refresh_tokens NULL	rotation chain, enables reuse detection (§15.7)

6.20 `instructor\_feedback`

Column	Type	Constraints	Notes
id	UUID	PK	
incident_id	UUID	FK → incidents NOT NULL	
instructor_id	UUID	FK → users NOT NULL	
rubric_overrides	JSONB	NULL	per-item score adjustments, preserved distinctly from the original automated scores row (§2.15)
comment	TEXT	NULL	
reopened_session	BOOLEAN	NOT NULL DEFAULT false	
created_at	TIMESTAMPTZ	NOT NULL	

Deliberately append-only and additive: an instructor's feedback never overwrites `scores`; the results UI (§2.12) displays both the automated score and any instructor feedback as distinct, attributed entries, preserving the pedagogical value of showing a Student both machine and human assessment.

6.21 Learning Platform Tables

Covered inline above (§6.18) alongside `certificates` since they share a domain and are small reference/join tables: `courses` (`id`, `title`, `description`, `career_track` `ENUM(soc_analyst,blue_team)`), `learning_paths` (`id`, `course_id` FK, `title`, `pass_threshold_percent`), `learning_path_scenarios` (`learning_path_id` FK, `scenario_id` FK, `sort_order`), `achievements` (`id`, `code`, `name`, `description`, `criteria` JSONB — e.g. "complete 5 ransomware scenarios with >90% score"), `user_achievements` (`user_id` FK, `achievement_id` FK, `earned_at`).

6.22 `audit\_logs`

Platform-wide, security/compliance-relevant action log — distinct in *purpose* from `investigation_actions` (§6.13, which is pedagogical/replay data scoped to a scenario session) even though both are append-only event logs.

Column	Type	Constraints	Notes
id	UUID	PK	
actor_user_id	UUID	FK → users NULL	NULL for system-initiated actions
actor_ip	INET	NULL	
action	TEXT	NOT NULL	e.g. login, login_failed, role_changed, password_reset, verdict_submitted, admin_impersonation_started, cohort_created, sso_config_changed
target_type	TEXT	NULL	
target_id	UUID	NULL	
metadata	JSONB	NULL	
correlation_id	UUID	NULL	ties back to the originating API request (§5.2)
occurred_at	TIMESTAMPTZ	NOT NULL	

`audit_logs` rows are **never updated or deleted** by application code (enforced by revoking `UPDATE/DELETE` grants on this table from the application's database role, leaving only `INSERT/SELECT` — a defense-in-depth measure against a compromised application server tampering with its own audit trail, §15.9).

6.23 Data Retention & Partitioning Strategy

Retention:

- `investigation_sessions` and all synthetic telemetry tables (§6.10–§6.12): retained in "hot" (immediately queryable) storage for **90 days** after `submitted_at/expires_at`, then the session's telemetry rows are exported as a compressed JSON bundle to object storage (`archived-sessions` bucket) and hard-deleted from Postgres. `scores`, `evidence_collection`, `analyst_notes`, and the incident's final report snapshot are **never archived-away** — they are small, permanently valuable records kept indefinitely (subject to account-deletion requests, §15.12). This split (bulky synthetic telemetry expires; compact scoring/report data persists forever) is what keeps the primary database's storage footprint proportional to *active* usage rather than all-time cumulative usage (§20).
- `audit_logs`: retained a minimum of **2 years**, aligned with common institutional compliance-training record-keeping expectations for the education/enterprise buyer segments (§1.3); never bulk-deleted by an automated job.
- `investigation_actions`: same 90-day hot-storage horizon as telemetry, archived alongside its session.

Partitioning: `security_events`-family tables (§6.12) and `audit_logs` are designed from MVP as **range-partitioned by month** on `occurred_at` (Postgres native declarative partitioning), even though MVP volume

does not strictly require it (§3.3). This is specified up front — rather than retrofitted later — because: (1) partition-per-month makes the 90-day archival job in the previous paragraph a cheap `DETACH PARTITION + export` instead of a slow row-by-row `DELETE`, and (2) retrofitting partitioning onto a live, large table is a materially riskier migration than starting partitioned. New monthly partitions are created by the scheduled job in §5.12, one month ahead of need.

6.24 Migration Tooling & Change Management

All schema changes go through a versioned migration tool (e.g., Prisma Migrate, Knex, or Flyway — any tool satisfying the requirements below; not prescribed further since this is an implementation-detail choice, §5.1) under these rules: migrations are **forward-only** in production (a bad migration is fixed by a new forward migration, never by editing/rolling back an already-applied one, matching the git-commit philosophy in this document's operating instructions); every migration is reviewed and applied through CI/CD (§5.1), never run ad hoc against production; destructive migrations (column/table drops) require a two-step process — stop writing to/reading the column in application code and deploy that first, *then* drop the column in a subsequent migration at least one release later — so a rollback of the application code never references a already-dropped column.

7. Telemetry Generator

7.1 Purpose and Design Philosophy

The Telemetry Generator is the service that makes SOCVerse possible without a single byte of real customer log data. Its job is to take a scenario's authored `ground_truth_definition` (§6.7) and produce a complete, internally consistent synthetic dataset — identities, devices, mailboxes, and thousands of individual events across the tables in §6.12 — such that: (a) the attacker's actions are present and discoverable in the data, (b) the data is surrounded by enough plausible, non-malicious "noise" that finding the attacker's actions requires actual investigation rather than pattern-matching an obviously-different-looking row, and (c) every field looks like it came from the real log source it imitates (a Windows Security Event Log 4624, a Microsoft 365 sign-in log entry, an Nginx access log line), because the *format* fluency is itself part of what a Student is learning.

This is fundamentally a **narrative-to-data compiler**, not a log replay tool and not a purely random generator. Two failure modes are explicitly designed against:

- **Pure randomness** produces data with no coherent story, making investigation feel like solving a puzzle with the wrong shape (SOC work is never "spot the row that looks different"; it's "follow a chain of cause and effect"). This is why the generator is narrative-driven rather than statistically-driven-from-scratch.
- **Hand-scripted, fully deterministic data** (the same 5,000 rows every time) makes scenarios trivially memorizable/shareable between Students, destroying both the assessment validity and the replayability that the subscription model depends on (§1.7). This is why every session gets its own `seed` (§6.9) driving procedural variation layered on top of the fixed narrative.

7.2 Generation Pipeline

For a given `investigation_session`, generation proceeds in five deterministic stages, all driven by the session's stored `seed` so the exact same output can be regenerated for debugging or instructor review:

16. **World seeding.** Instantiate the `identities`, `devices`, and mailbox population defined by the scenario's `ground_truth_definition`, plus a configurable number of *decoy* identities/devices with no role in the attack narrative (typical ratio: 70–90% decoy population, 10–30% narrative-relevant, tunable per scenario difficulty — higher difficulty means a larger haystack, §12.2). Decoy population attributes (names, departments, device hostnames) are drawn from curated name/word lists with the seeded RNG, not an LLM call, to keep this stage fast and free of runtime external dependencies (§7.6).
17. **Baseline behavior synthesis.** For every identity and device (decoy and narrative-relevant alike), generate a plausible baseline of "normal" activity across the scenario's simulated time window (typical window: 24–72 hours) — normal business-hours sign-ins from the identity's `home_country`, routine software/process activity, ordinary DNS/HTTP traffic. This baseline is what makes the dataset *realistic* rather than sparse, and it is what conditional-access/impossible-travel/anomaly detection logic (§8) actually has to differentiate against.
18. **Ground-truth event injection.** Walk the scenario's authored kill-chain steps (an ordered list of `{technique, entity, event_template, relative_timestamp}` tuples in

ground_truth_definition) and materialize each into one or more concrete rows in the appropriate §6.12 table, each flagged `is_ground_truth_evidence = true` and tagged with its `mitre_technique_id`. Timestamps are placed relative to the baseline (e.g., "3 hours after the phishing email is opened" rather than an absolute clock time), so the same narrative structure produces a different absolute timeline every session.

19. **Correlation stitching.** Assign shared `correlation_ids` (§6.12) across events that represent one causal chain — e.g., the phishing email's `email_messages` row, the resulting malicious `process_events` row on the victim's device, and the `network_events` row for its C2 callback all share a `correlation_id` — so the Global Timeline's (§2.9) cross-entity correlation lines have real structure to surface, and so the Scoring Engine (§12.4) can evaluate whether a Student's timeline construction reflects the true causal chain, not just a list of individually-correct events.
20. **Noise injection.** Add a configurable volume of look-alike-but-benign events (a failed sign-in from a typo'd password, a legitimate admin RDP session, a benign PowerShell script run by IT) specifically chosen per scenario to be plausible false-positive bait — this is what prevents "everything flagged as evidence is malicious" from being a viable non-investigative strategy, and it is the direct data source for the Scoring Engine's false-positive-rate component (§6.14, §12.4).

7.3 Realism Techniques Per Log Source

- **Windows Security/Sysmon-style events** (`process_events`, `registry_events`, `sign_in_events` for domain-joined narratives): field vocabulary (event structure, integrity levels, standard LOLBin process names like `powershell.exe/rundll32.exe/certutil.exe` used in both benign IT-admin and malicious contexts) is drawn from a curated reference table of realistic Windows artifact patterns maintained by the content team (§12.1), not invented ad hoc per scenario — ensuring consistency of "what right looks like" across the whole scenario library.
- **Linux logs:** `auth.log/syslog`-style structure for `sign_in_events/process_events` on `os_platform = linux` devices — SSH key vs. password auth distinction, `sudo` invocation patterns — supporting cloud/server-focused scenarios (§1.8).
- **Microsoft 365 / Azure-style sign-ins:** `sign_in_events` fields (`client_app`, `is_legacy_auth`, `conditional_access_evaluations`) are modeled on the public schema of Entra ID sign-in logs closely enough for pattern-transfer (§1.6) without reproducing Microsoft's actual product UI or proprietary documentation text.
- **Email events:** full RFC 5322-shaped header block in `headers_raw` including `Received:` chain hops (with plausible relay hostnames/IPs), `Authentication-Results:` header reflecting the stored SPF/DKIM/DMARC verdicts, and `Message-ID`, so the Email Portal's raw-header view (§11.2) is genuinely investigable rather than a summary.
- **Firewall/VPN/Proxy/DNS:** field vocabulary modeled on common open log formats (Cisco ASA-style, syslog RFC 5424) for firewall/VPN, and standard DNS query/response structure — chosen because these are widely-documented public formats appropriate to imitate, not any single vendor's proprietary format.
- **Cloud control-plane events:** `cloud_events.action_name` values modeled on well-known, publicly documented cloud API action names (e.g., IAM key creation, storage bucket policy changes)

representative of common cloud-attack patterns, without targeting any single provider's exact API surface, keeping the scenario category provider-agnostic.

7.4 Volume and Difficulty Scaling

Each scenario's `ground_truth_definition` specifies target volumes per table (typical range 5,000–50,000 total events for beginner, up to 100,000–250,000 for expert, §3.9's cap), and a **signal-to-noise ratio** target. Difficulty (§12.2) is primarily a function of: population size (more decoy entities to search through), signal-to-noise ratio (rarer ground-truth events relative to noise), narrative directness (how many investigative hops separate an alert from the full picture), and false-positive bait density — **not** of UI complexity, which stays constant across difficulty levels so a Student's growing competence is measured, not their growing familiarity with a harder interface.

7.5 Randomization Without Losing Ground Truth

Per-session variation (different absolute timestamps, different decoy names/hostnames/IPs, different specific noise events, and — for higher-difficulty scenarios — random selection from a pool of 2–4 authored "distractor" sub-narratives that get mixed into the noise, §12.2) means no two Students, and no two attempts by the same Student, see an identical dataset. What never varies within a scenario version is the **kill-chain structure and required technique set** — the actual thing being assessed — which is what keeps automated scoring valid across infinite procedural variation: the scoring engine (§12.4) evaluates against the abstract `ground_truth_definition`, not against a specific generated dataset.

7.6 No Runtime LLM Dependency for Core Generation

Core telemetry generation (§7.2–§7.5) is a deterministic, seeded procedural process with **no runtime call to any external LLM API** — this is a deliberate cost and reliability decision (§20): generation must complete within the latency budget in §3.1 even under load, must be free of third-party API cost per session, and must be exactly reproducible from a stored seed for debugging/instructor-review purposes, none of which are compatible with a live generative-AI call in the critical path. Where natural-language content varies (email body phrasing, decoy document text), the generator selects and lightly parameterizes from an authored template library (§12.1) rather than generating prose live. LLM-assisted **content authoring** (an author-time tool that helps a human write new scenario templates and telemetry) is a distinct, offline, human-in-the-loop capability described in §14.5 — it is never in the request path of a live scenario session.

7.7 Pre-Generation and Caching

Because cold-start latency (§3.1) matters and because generation is CPU-bound work best kept off the interactive request path entirely, telemetry is **not** generated synchronously when a Student clicks "Start Scenario." Instead:

- A pool of pre-generated telemetry bundles is maintained per **popular** scenario version (generated by a background job, §5.9, triggered on scenario publish and replenished as the pool is consumed), stored in object storage (`pregenerated-telemetry` bucket, §5.11) as a serialized bundle keyed by `scenario_version_id` + a fresh seed, ready to be bulk-loaded into Postgres in a single transaction when a

Student starts a session — this turns "cold start" into "load a pre-built bundle," which is what makes the aggressive §3.1 startup-latency targets achievable on modest MVP hardware.

- For lower-traffic/long-tail scenarios where maintaining a standing pool isn't warranted, generation happens synchronously in the `telemetry-generation` queue (§5.9) with the Student shown a short, honest loading state; the pipeline in §7.2 is engineered to complete this synchronous path within the §3.1 budget for the default volume ranges in §7.4.
- Pool replenishment and pre-generation are themselves subject to the same idempotent-job design (§5.9, §3.8) as every other background job.

8. Alert Engine

8.1 Purpose and Detection Model

The Alert Engine consumes the telemetry a session's Telemetry Generator run produced (§7) and materializes the alerts (§6.15) a Student actually sees on the Alert Dashboard (§2.1). It is deliberately **not** a general-purpose, user-authorable detection-rules engine (that would make SOCVerse a SIEM-authoring product, contradicting §1.6) — it is a fixed library of proprietary `detection_rules` (§6.15), each an internally-implemented function (not a Student-facing query language) that evaluates a specific, well-understood detection pattern against a session's event tables and emits an alert when the pattern matches.

Because the Telemetry Generator already knows and flags every ground-truth-evidence event (`is_ground_truth_evidence`, §6.12) and every intentional noise/false-positive-bait event, the Alert Engine's rule library is authored to be **behaviorally realistic** — a rule fires because a real analytic pattern is present in the data (an impossible-travel geo/time delta, a rare parent-child process pairing, an SPF-fail-plus-lookalike-domain combination) — rather than simply "fire an alert on every `is_ground_truth_evidence` row." This distinction matters pedagogically: it means alerts sometimes fire on noise (realistic false positives, §8.6) and some ground-truth evidence is deliberately **sub-alert-threshold** (discoverable only through manual investigation, not handed to the Student by an alert), which is what makes the Search, Global Timeline, and manual pivoting features (§2.8–§2.9) genuinely necessary rather than decorative.

8.2 Rule Categories

The MVP rule library covers, at minimum, one or more concrete rules per attack category in §1.8:

Category	Example rules
Identity	impossible travel (geo-distance/time-delta threshold), password spray (N failed sign-ins across M distinct identities from one source within a window), MFA fatigue (repeated MFA prompts followed by an approval), legacy-auth sign-in from a risky location, sign-in from a newly-seen ASN/country for that identity
Endpoint	rare parent-child process pairing (e.g., <code>winword.exe</code> → <code>powershell.exe</code>), known-LOLBin invocation with suspicious command-line flags, new persistence artifact (registry Run key / scheduled task creation outside a maintenance window), mass file-modify-then-rename matching a ransomware pattern, USB mass-storage connect followed by high-volume file copy
Email	SPF/DKIM/DMARC failure combined with a display-name/domain lookalike, first-time-sender-to-org plus urgency-language subject heuristic (matched against an authored keyword/pattern list, not live NLP inference, §8.1), known-malicious attachment hash/URL match against <code>threat_intel_indicators</code>
Network/C2	periodic beaconing interval detection (regular inter-request timing to one external host), DNS query

Category	Example rules
	volume/entropy anomaly (DGA-style domain pattern), connection to a <code>threat_intel_indicators</code> -listed IP/domain
Cloud	anomalous control-plane action from a non-baseline location/identity, storage resource made public, new access key created for a privileged identity outside baseline behavior
Privilege escalation / lateral movement	sign-in to an unusual number of distinct devices in a short window by one identity, privileged-group membership change, use of a service account interactively

Each rule is implemented as a scoped SQL/query-layer function operating only against the triggering session's own tables (never cross-session), executed by the Alert Engine worker (§5.9) once telemetry generation for that session completes.

8.3 Correlation Logic

Beyond single-event rules, a smaller set of **correlation rules** operate over the `correlation_id` links the Telemetry Generator stitched in (§7.2 stage 4): if two-plus individually-triggering conditions share a `correlation_id` (e.g., a risky sign-in and a subsequent mass file-encryption event on a device that sign-in accessed), the Alert Engine emits them as **linked alerts** (a `related_alert_id` self-reference on `alerts`, surfaced in the UI as "related alerts" on the alert detail view) rather than fully independent items — modeling how real SOC tooling nudges an analyst toward recognizing one incident instead of N unrelated alerts, without pre-building the incident for the Student (that synthesis remains their graded task, §2.2).

8.4 Alert Generation Timing

For standard (non-live-mode) sessions, all alerts for the full simulated time window are generated in one batch immediately after telemetry generation completes, queued as `alert-correlation` (§5.9), chained after the `telemetry-generation` job via the queue's job-dependency feature so a session never shows a Student a partially-alerted, inconsistent state. For **live/time-phased** scenarios (§12.7), the Alert Engine still evaluates the full dataset up front but *releases* alerts to the Student incrementally, matched to `current_scenario_time` (§6.9) as it advances, pushed over the WebSocket channel (§4.4, §5.10) — this supports scenarios designed to be worked in real time (e.g., "you have 30 simulated minutes before the ransomware detonates") without requiring a second, separate detection implementation.

8.5 Severity Assignment

Each `detection_rules` row carries a `default_severity`; the Alert Engine may adjust severity per-instance using simple, documented modifiers (e.g., +1 severity tier if the primary entity `is_privileged`, §6.10) so severity is not purely static per rule type — mirroring how real detection content commonly weights privileged-account activity higher. Severity directly informs default Alert Dashboard sort order (§2.1) but never gates alert visibility — every generated alert is visible to the Student regardless of severity, since under-triaging low-severity alerts is itself a realistic and gradable failure mode.

8.6 False Positives (By Design)

A configurable subset of rules are permitted to fire on `is_ground_truth_evidence = false` (noise) events when the underlying pattern genuinely matches — these are marked `is_false_positive_by_design = true` (§6.15) at generation time and are the direct mechanism behind the false-positive-handling component of scoring (§6.14, §12.4: correctly dismissing these, with a substantive dismissal reason, is rewarded; escalating them into an incident is penalized). This is authored deliberately per scenario (the content team chooses which noise events are "realistic enough to trip a rule") rather than being an accidental/uncontrolled side effect of the rule logic, keeping false-positive rate a tunable difficulty and pedagogical parameter rather than a data-quality bug.

8.7 Deduplication and Suppression

- **Deduplication:** repeated instances of the same rule firing for the same primary entity within a configurable rolling window (e.g., 10 failed sign-ins each individually matching a "failed sign-in" pattern) are collapsed into a single `alerts` row with `dedup_count` incremented and `last_seen_at` extended, rather than flooding the queue with near-duplicate alerts — mirroring standard SIEM alert-fatigue mitigation and keeping the Alert Dashboard realistically sized (tens, not thousands, of alerts per session even at high event volume, §7.4).
- **Suppression:** an authored per-scenario suppression list can mark specific known-benign patterns (e.g., a scheduled backup job's routine large network transfer) as fully suppressed (never surfaced as an alert at all, though still present and searchable in raw telemetry) — used sparingly, and only where the content team judges that surfacing it as a dismissible alert would add tedium rather than pedagogical value; the default is to let a realistic false positive appear and be dismissed (§8.6), not to suppress it.

8.8 Alert Prioritization on the Dashboard

Default sort is severity descending, then recency descending, matching common SIEM/XDR triage-queue conventions so the ergonomic transfer (§1.6) holds; Students may re-sort/filter freely (§2.1). No proprietary "risk score" ranking algorithm beyond severity + recency is introduced at MVP — keeping the prioritization model transparent and explainable is itself pedagogically important (a Student should be able to reason about *why* something is prioritized, not trust an opaque score), and is revisited only if usage data shows Students need finer-grained triage guidance at higher difficulty tiers.

9. Identity Investigation Service

9.1 Purpose

The Identity Portal is SOCVerse's proprietary analog to an Entra ID-style identity investigation surface (§1.6) — it is where a Student investigates *who* was involved in an incident: sign-in behavior, risk signals, group membership, and device associations. It is backed entirely by the `identities` and related tables in §6.10 and the `sign_in_events/conditional_access_evaluations` tables in §6.12.1, scoped to the active `session_id`.

9.2 Identity Directory (List View)

Searchable/filterable list of every `identities` row in the session (decoy and narrative-relevant alike — the Student never sees the internal `is_ground_truth_actor` flag, §6.10). Columns: display name, department, risk level, MFA status, account status. Filters: risk level, department, MFA status, "has risky sign-ins" toggle. This view intentionally surfaces the full decoy population (§7.2) so that finding the *right* identity to focus on is part of the investigative task, not a pre-filtered given.

9.3 Identity Profile

Per-identity detail page assembling:

- **Overview panel:** name, UPN, department, job title, manager (linked to the manager's own profile via `manager_identity_id`), account status, MFA status, risk level.
- **Devices panel:** all devices where `primary_identity_id` matches, or where a `sign_in_events` row links this identity to that device — cross-links directly into the Device Portal (§10).
- **Group membership panel:** from `identity_group_memberships`, flagging `role_assignable` groups distinctly since privileged-group membership is frequently narrative-relevant.
- **Conditional Access panel:** `conditional_access_evaluations` rows for this identity, showing policy name, result, and reasons — this is where a Student sees *why* a sign-in was granted, blocked, or challenged for MFA, mirroring the investigative value of a real CA evaluation log.
- **Password History (simulated):** a synthetic, non-reversible history of password-change *events* (`password_changed_at`, `changed_by` — self or admin-reset, reason if flagged) — modeled as metadata only; **no actual password values, hashes, or anything resembling real credential material are ever stored or displayed for synthetic identities**, both because it adds no investigative value and to avoid any pattern that could be mistaken for real-credential handling guidance.

9.4 Sign-in Timeline

The core investigative surface of the Identity Portal: a chronological list of every `sign_in_events` row for this identity, each rendered with source IP, geo (country/city), application, result, legacy-auth flag, and a computed "distance from previous sign-in" annotation used for impossible-travel reasoning. Filterable by result (success/failure/blocked) and by a "risky only" toggle that surfaces sign-ins the Alert Engine (§8.2) flagged.

9.5 Risky Sign-ins

A dedicated filtered view (not a separate table — computed by joining `sign_in_events` against alerts whose `primary_entity_id` is this identity and whose `detection_rules` category is identity-related, §8.2) surfacing exactly the sign-ins that triggered identity-category detections, with the specific triggering rule named — giving the Student a starting point without doing their investigation for them (they still must determine relevance/severity themselves).

9.6 Impossible Travel Detection (Investigative Surface)

The Alert Engine's impossible-travel rule (§8.2) computes great-circle distance between consecutive sign-in geo-points for the same identity and flags pairs whose implied travel speed exceeds a realistic threshold (default: exceeds commercial-flight-speed feasibility with a configurable buffer for VPN/mobile-network geo-imprecision, tunable per scenario to control difficulty — a tighter buffer surfaces more borderline cases). The Identity Profile visually pairs flagged sign-in events (e.g., a "previous sign-in" reference chip) so a Student can see both ends of the anomalous pair without manually cross-referencing timestamps by hand, while still requiring them to judge severity and write the justification themselves.

9.7 Password Spray Detection (Investigative Surface)

Surfaced at the *source-IP* level rather than per-identity: the Search feature (§2.8) and a dedicated "Authentication Investigation" cross-identity view let a Student query "all failed sign-ins from source IP X" to discover the fan-out across many identities that characterizes spraying — the Alert Engine's password-spray rule (§8.2) pre-flags the pattern as an alert, but confirming which specific identities were targeted and whether any succeeded (indicating a compromised account requiring escalation) is manual investigative work using this cross-identity query capability.

9.8 Geo-location and Behavior Timeline

Every sign-in's `source_country/source_city` is rendered on a simple map/list visualization in the profile (no external mapping API call — city/country-to-coordinate lookup uses a small bundled static reference dataset, keeping this dependency-free per §7.6's no-runtime-external-call principle). The "Behavior Timeline" referenced in the product brief is realized as the same chronological Sign-in Timeline (§9.4) combined with any `investigation_actions` (§6.13) the Student themselves has taken against this identity (e.g., "you disabled this account at 14:32") rendered inline, so the identity's own timeline and the Student's response actions are visible together.

9.9 What Is Never Exposed to the Student

Per §6.10/§6.12, the following columns are stripped by the API-layer DTO mapping (§16) before any Identity Portal response leaves the service, regardless of role (even Instructor Mode's "review" view uses a separate, explicitly-ground-truth-aware endpoint, §2.15, gated to instructor/admin roles only):

`is_ground_truth_actor`, `is_ground_truth_evidence`, `mitre_technique_id` on event rows, and the internal `seed/ground_truth_definition` of the owning session/scenario. This separation is enforced by a dedicated response-serialization layer (§18.3), not by convention alone, since a leaked ground-truth flag would trivially break the assessment validity of every scenario built on that data.

10. Device Investigation Service

10.1 Purpose

The Device Portal is SOCVerse's proprietary analog to a Defender for Endpoint-style investigation surface (§1.6) — where a Student investigates *what happened on a machine*: process execution, persistence, network activity, and file activity. Backed by `devices` (§6.11) and the device-scoped event tables in §6.12 (`process_events`, `file_events`, `network_events`, `dns_events`, `registry_events`, plus `installed_software`, `device_services`, `startup_entries`, `usb_events`).

10.2 Device Overview

Directory list (mirrors §9.2's pattern): hostname, OS platform/version, primary identity, risk level, isolation status, last-seen. Profile overview panel adds: full installed-software inventory (`installed_software`), running-services list (`device_services`), and startup-entries list (`startup_entries`) — the three "what is configured to run on this box" inventories a real endpoint investigation starts from.

10.3 Process Tree

The signature investigative view of the Device Portal: `process_events` rows for the device rendered as a navigable tree via `parent_process_guid` self-reference (§6.12.2), each node showing image path, command line, hash, integrity level, and the identity context it ran under. Nodes corresponding to alert-triggering or ground-truth-adjacent activity are not visually distinguished from any other node by default (that would leak the answer) — a Student must expand and read command lines to judge suspiciousness, exactly as in real endpoint tooling. A "search within tree" control lets a Student filter the tree to nodes matching a command-line substring or hash, without collapsing the surrounding structure needed to reason about parent/child relationships.

10.4 Network Connections and Open Ports

`network_events` rows for the device, filterable by direction, remote IP/port, and process (cross-linked from a specific process-tree node's "show network activity" action, joining on `process_guid`). "Open ports" is realized as the set of distinct `local_port` values with `direction = inbound` observed for the device within the session window — a derived view over `network_events`, not a separately-modeled point-in-time port-scan table, since the investigative question ("what has this device been listening on / connecting to") is adequately answered by the event history without a redundant live-state model.

10.5 File Timeline

`file_events` rows for the device in chronological order, filterable by action type (created/modified/deleted/renamed/encrypted). The `encrypted` action type is used specifically by ransomware-category scenarios (§1.8) to let a Student observe and quantify the blast radius (count and paths of encrypted files, encryption rate over time) directly from the timeline, which also feeds the incident-response containment-timing rubric (§2.20, §12.4 — did the Student isolate the device before or after a meaningful fraction of files were encrypted in the simulated timeline).

10.6 Registry Changes

`registry_events` rows, filterable by action and key path, with a dedicated "persistence-relevant" quick filter (a curated list of well-known persistence key-path prefixes — Run/RunOnce keys, Winlogon Shell/Userinit, service ImagePath — matched at query time, not a separate flag column) that also cross-references `startup_entries` so a Student can confirm whether a registry change actually resulted in a live startup entry.

10.7 USB Activity

`usb_events` rows (connect/disconnect, device serial, volume label) correlated in the UI (via matching `correlation_id`, §7.2) with any `file_events` involving that volume — directly supporting insider-threat and data-exfiltration scenario categories (§1.8) where USB mass-storage use is the exfiltration vector.

10.8 Security Events (Device-Scoped Summary)

A consolidated, cross-table chronological feed (process/file/network/DNS/registry events for this device interleaved by timestamp) is exposed as a single "Device Timeline" tab, functioning as the device-scoped equivalent of §2.9's Global Timeline — useful because real endpoint investigation is rarely confined to one artifact type at a time, and forcing a Student to tab between five separate lists for a single device would work against the investigative flow this feature exists to teach.

10.9 Malware History

Derived view: any `file_events`/`process_events` rows whose `hash_sha256` matches a `threat_intel_indicators` row with `reputation = malicious` (§6.15, §2.7), joined and presented as a "known-malicious activity on this device" panel — this is a query-time join, not a separately maintained table, keeping a single source of truth for indicator reputation.

10.10 Device Risk

`devices.risk_level` is computed the same way as `identities.risk_level` (§8, §9): a function of how many and how severe the alerts (§6.15) whose `primary_entity_type = 'device'` and `primary_entity_id` match this device are, recomputed by the Alert Engine whenever new alerts are generated for the session. It is a coarse triage signal shown in list/overview views, never a substitute for the Student's own judgment on any individual profile page — no page in the Device Portal states a verdict on the device's behalf.

10.11 Isolation Status and the Isolate Action

`isolation_status` (§6.11) is mutated by the Student-initiated "Isolate Device" action (§2.20), recorded as an `investigation_actions` row (`action_type = 'isolate_device'`, §6.13) with a timestamp compared, at scoring time, against the simulated timeline of the attack's progression on that device (§12.4) — isolating early enough to plausibly limit blast radius is a distinct, positively-weighted rubric component from simply identifying that the device was compromised at all, reflecting that real SOC work is graded on response, not just detection.

10.12 What Is Never Exposed to the Student

Identical policy to §9.9: `is_ground_truth_actor`, all `is_ground_truth_evidence/mitre_technique_id` fields on event rows, and `installed_software.is_known_bad` are stripped at the serialization layer (§18.3) for every Student-facing Device Portal endpoint.

11. Email Investigation Module

11.1 Purpose

The Email Portal is SOCVerse's proprietary analog to an Outlook/Defender for Office 365-style message investigation surface (§1.6) — the primary surface for phishing and Business Email Compromise (BEC) scenario categories (§1.8). Backed by `email_messages`, `email_attachments`, and `email_urls` (§6.12.9), scoped to the active `session_id`.

11.2 Mailbox/Message Search

List view of `email_messages` for the session, filterable by direction, sender, subject keyword, SPF/DKIM/DMARC result, and "has attachment"/"has URL" toggles — supporting the same field=value search ergonomics as the platform-wide Search feature (§2.8), scoped specifically to email fields for faster in-portal triage.

11.3 Email Viewer

Two-pane message reader: a rendered view (`body_html`, sandboxed/sanitized for safe display — see §11.7) styled to resemble a familiar mail-client reading pane, and a "View Source"/raw-headers toggle exposing `headers_raw` in full RFC 5322-shaped text, including the synthetic `Received:` relay chain, `Message-ID`, and `Authentication-Results:` header — mirroring the real investigative habit of checking raw headers when the rendered view is insufficient or suspected of being spoofed.

11.4 Authentication Results (SPF/DKIM/DMARC)

Dedicated panel summarizing `spf_result`, `dkim_result`, and `dmARC_result` in plain-language form (e.g., "SPF: Fail — sending IP not authorized for this domain") alongside the raw `Authentication-Results:` header line, so a Student learns to read both the human-readable summary and the raw form real tooling actually presents. BEC scenarios (§1.8) specifically exercise the case where authentication passes cleanly (a compromised legitimate mailbox, or a convincing lookalike domain with its own valid SPF/DKIM) — the panel never itself declares a verdict; it only reports the mechanical result, keeping the "is this actually malicious" judgment the Student's task.

11.5 Attachment Analysis

`email_attachments` rows for the open message, each showing filename, content type, size, hash, and `sandbox_verdict`. Clicking a hash cross-links to the Threat Intelligence lookup (§2.7) for full indicator context. **No real file content is ever generated, stored, or served** — attachments are metadata-only records; "sandbox_verdict" is an authored/generated field, not the output of an actual sandboxing process, consistent with the platform never handling or executing any real (or even realistic-but-functional) malicious code (§15.11's secure-content principle).

11.6 URL Analysis

`email_urls` rows for the open message: displayed URL vs. actual target (supporting lookalike-domain and link-mismatch investigation), `reputation` (from the same scale as §2.7), and `is_rewritten_by_safe_links` (modeling the common enterprise pattern of URL-rewriting security gateways, so Students learn to recognize and unwrap rewritten links). Clicking through **never navigates anywhere** — target URLs are inert display strings rendered as plain text/copy-to-clipboard, not live anchors, eliminating any possibility of a Student's browser actually requesting an external (even if internally-hosted-and-safe) address from within a graded investigation flow.

11.7 Sandbox Results

Presented as part of the Attachment Analysis panel (§11.5) rather than a separate service: `sandbox_verdict` plus an authored free-text "behavioral summary" field (e.g., "Dropped and executed a secondary payload; established outbound connection to 45.x.x.x on port 443") giving the Student sandbox-report-style narrative detail to correlate against the Device Portal's process/network timeline for the recipient's device — this cross-portal correlation (email → device) is one of the primary "connect the dots across consoles" learning objectives the whole multi-console design exists to teach (§1.2).

11.8 Campaign Correlation

A "Similar Messages" panel on the message detail view surfaces other `email_messages` in the same session sharing sender domain, a common `correlation_id` (§7.2), or overlapping `email_urls/email_attachments` hashes — supporting scenarios where a phishing campaign targets multiple identities and the Student must recognize the pattern across mailboxes, not just assess one message in isolation (directly relevant to BEC and broader campaign-style scenario narratives, §1.8).

11.9 Threat Intelligence Integration

Every hash (`email_attachments.hash_sha256`) and URL/domain (`email_urls.url`) is resolvable through the same `threat_intel_indicators` lookup used platform-wide (§2.7, §6.15) — the Email Portal does not maintain a separate reputation system, keeping "what is this indicator's reputation" a single, consistent answer regardless of which portal a Student encountered it from first.

11.10 What Is Never Exposed to the Student

Identical policy to §9.9/§10.12: `email_messages.is_ground_truth_evidence` and `mitre_technique_id` are stripped at the serialization layer (§18.3). Additionally, because `body_html` is rendered client-side, it is sanitized server-side (strip `<script>`, event handlers, and any active content, §15.6) before ever leaving the API — defending against the theoretical case of a maliciously-authored scenario template accidentally including unsafe markup, not because any real inbound content is untrusted (all content is authored by SOCVerse's own content team, §12.1).

12. Scenario Engine

12.1 Scenario Builder and Ground-Truth Definition Schema

The Scenario Builder is the authoring tool (available to Platform Admins at MVP launch; opened to vetted Instructors in Phase 2, §1.7, §2.15) used to produce a `scenario_versions.ground_truth_definition` (§6.7) JSON document. Its schema, in outline:

```
{
  "metadata": { "category", "difficulty", "estimated_minutes", "narrative_summary" },
  "population": {
    "narrative_identities": [ { "role": "victim|attacker_persona|bystander", "attributes":
{...} } ],
    "narrative_devices": [ {...} ],
    "decoy_population_size": { "identities": N, "devices": N },
    "world_time_window_hours": N
  },
  "kill_chain": [
    {
      "step_order": 1,
      "mitre_technique_id": "T1566.001",
      "entity_ref": "victim_identity_1",
      "event_template_id": "phishing_email_generic_invoice_v3",
      "relative_timestamp": "+0h",
      "correlation_group": "phish-chain-1",
      "is_required_for_full_credit": true
    },
    ...
  ],
  "noise_profile": {
    "signal_to_noise_ratio": 0.02,
    "false_positive_bait": [ { "event_template_id": "...", "count": N } ]
  },
  "distractor_pool": [ { "sub_narrative_id": "...", "inclusion_probability": 0.3 } ],
  "scoring_rubric": {
    "required_techniques": [ "T1566.001", "T1204.002", "T1071.001" ],
    "required_verdict": "true_positive",
    "min_evidence_items": 4,
    "containment_expectations": [ { "action": "isolate_device", "target": "victim_device_1",
"by_relative_time": "+3h", "weight": 0.15 } ]
  },
  "hints": [ { "unlock_cost_percent": 5, "text": "...", ... } ]
}
```

Event templates (`event_template_id`) reference the curated, realism-vetted template library maintained by the content team (§7.3) — an author picks *which* templates and *where* they sit in the kill chain, rather than hand-writing raw log rows, which is what keeps scenario authoring throughput reasonably high and keeps every scenario's underlying data realistic by construction. The Scenario Builder UI walks an author through this schema as a guided multi-step form (define narrative → place kill-chain steps on a visual timeline → configure noise/distractors → define rubric → define hints) rather than requiring hand-edited JSON, with the raw JSON view available as an "advanced" tab for power authors.

12.2 Difficulty Levels

Difficulty (beginner/intermediate/advanced/expert, §6.7) is realized as a bundle of the following authored parameters, not a single knob, so difficulty is meaningfully distinct in *kind* as well as degree:

Parameter	Beginner	Expert
Decoy population size	Small (10–20 identities/devices)	Large (100+)
Signal-to-noise ratio	Higher (evidence easier to find)	Lower
Kill-chain directness	Alerts point close to the answer	Alerts under-cover; more manual pivoting required
Distractor sub-narratives	None	1–3 competing plausible-but-wrong narratives mixed in
False-positive bait density	Low	High
Hint availability/cost	More hints, lower cost	Fewer hints, higher cost
Time pressure (live mode, §12.7)	Off or generous	On, tighter windows

This structure is what allows the same scenario *category* (e.g., ransomware) to be offered at multiple difficulty tiers as distinct `attack_scenarios` rows without duplicating content-authoring effort from scratch — a common pattern is to author one narrative once and produce a beginner and expert variant by adjusting only the population/noise/distractor parameters around the same fixed kill chain.

12.3 Hidden Ground Truth

The `ground_truth_definition`, the `is_ground_truth_evidence/is_ground_truth_actor/mitre_technique_id` columns on every synthetic entity/event table, and the `seed` are collectively "hidden ground truth" — accessible only to: the Telemetry Generator and Alert Engine (server-side workers, §7–§8), the Scoring Engine (§12.4), and Instructor/Admin-only review endpoints (§2.15, gated by role at the API Gateway and re-checked at the service layer, §15.2). No Student-facing endpoint, WebSocket message, or exported artifact (PDF report, certificate) ever includes these fields; this is enforced structurally by the DTO/serialization layer (§18.3) rather than left to per-endpoint discipline, and is covered by a dedicated automated test suite (§12.6) that fails the build if any Student-facing response schema includes a ground-truth field.

12.4 Automated Evaluation (Scoring Engine)

Runs as a background job (`scoring queue`, §5.9) triggered by scenario submission (§2.11), computing the `scores` row (§6.14) via five independently-weighted components (default weights shown; configurable per scenario via `scoring_rubric`, since some categories reasonably weight containment speed higher than others):

21. **Technique accuracy (30%):** overlap between the Student's `incident_techniques` tags and `scenario_techniques.is_required_for_full_credit = true`, penalizing both misses and incorrect additions (precision and recall both matter — tagging every possible technique "just in case" is not free).

22. **Evidence precision/recall (30%):** overlap between `evidence_collection` rows and the underlying `is_ground_truth_evidence = true` rows the Student's `event_id` references point at, computed against `min_evidence_items` and the full ground-truth evidence set.
23. **False-positive handling (15%):** correctly dismissed `is_false_positive_by_design` alerts (§8.6) with a substantive (non-empty, minimum-length) dismissal reason score positively; escalating them into an incident or pinning them as evidence scores negatively.
24. **Response/containment (15%):** `investigation_actions` compared against `containment_expectations` — was the right action taken, against the right target, within the expected relative-time window.
25. **Verdict correctness (10%):** did `incidents.verdict` match `scoring_rubric.required_verdict`.

The overall percentage is the weighted sum; the full itemized breakdown (not just the final number) is stored in `scores.rubric_breakdown` and is what the results screen (§2.12) renders, because showing *only* a final percentage would undermine the platform's core pedagogical premise — a Student needs to see specifically which evidence they missed and which technique tags were wrong to actually learn from an attempt, not just know they scored 61%.

12.5 Hints

Each scenario's `hints` array (§12.1) is an ordered, incrementally-revealing set of nudges (e.g., hint 1: "Check this identity's recent sign-in activity"; hint 2 references a specific alert; hint 3 references a specific technique). Requesting a hint is recorded as an `investigation_actions` row and applies `hint_penalty_percent` (§6.14) to the final score, deducted proportionally to the hint's authored `unlock_cost_percent` — this makes hints a legitimate, always-available strategy (never gated behind a paywall or limited count, since discouraging a stuck Student from getting unstuck at all would hurt retention far more than a modest score deduction) rather than a rare emergency-only feature.

12.6 Instructor Override and Scenario Versioning

Instructor override of a *finished session's grading* is covered in §2.15/§6.20 (`instructor_feedback`) — additive, never destructive to the automated score. Instructor override of the *scenario content itself* (for Instructor-authored custom scenarios, §1.7 Phase 2) follows the same `scenario_versions` append-only model as Platform Admin-authored content (§6.7): editing a published scenario creates a new `scenario_versions` row; `attack_scenarios.current_version_id` only repoints after the new version passes an automated **scenario validation pipeline** — a required pre-publish check verifying: every `required_technique` in the rubric has at least one corresponding kill-chain step, every referenced `event_template_id` exists, no Student-facing field path in the generated output schema contains a ground-truth field (§12.3's automated test, run per-scenario at publish time in addition to the platform-wide test suite), and a smoke-test session can be generated, alerted, and scored end-to-end without error. A scenario failing validation cannot be published, protecting both content quality and the "the backend always knows the answer" guarantee (§1.2) from author error.

12.7 Randomization and Live/Time-Phased Mode

Per-session randomization mechanics are specified in §7.5 (population/timestamp/noise variation) and §12.2 (distractor sub-narrative inclusion). **Live/time-phased mode** (`scenario_versions.ground_truth_definition.metadata` flag, referenced in §8.4) is an authoring option where alerts and evidence are released to the Student incrementally against `investigation_sessions.current_scenario_time` (§6.9) rather than all at once — driven by a scheduled advancement of `current_scenario_time` (a background job ticking the session's simulated clock forward at a configurable rate, e.g., 1 simulated hour per 2 real minutes) pushed to the client over WebSocket (§5.10). This mode is reserved for `advanced/expert` scenarios specifically designed around response-time pressure (e.g., ransomware-in-progress narratives, §1.8) — it is not the default mode, since forcing time pressure onto every scenario would undermine the reflective, self-paced learning experience that suits most of the platform's use cases (§1.4).

13. Learning Platform

13.1 Courses and Labs

A `courses` (§6.21) row is a thematic grouping of content (e.g., "Identity Threat Investigation Fundamentals") tagged to a `career_track`. Each course contains one or more `learning_paths`, and each learning path is an ordered sequence of `attack_scenarios` (via `learning_path_scenarios`, §6.21) — "labs" in the product brief's terminology map directly onto individual scenario attempts (§6.9's `investigation_sessions`) taken in the context of a learning path, not a separate content type; this avoids maintaining two parallel content models (standalone labs vs. scenarios) for what is pedagogically the same underlying unit of work.

13.2 Progress Tracking

Per-user, per-learning-path progress is a derived view (not a separately stored table at MVP) over `investigation_sessions` + `scores`, computed as: which `learning_path_scenarios` have at least one session meeting `pass_threshold_percent` (§6.21), in what order they were completed, and best/most-recent score per scenario. This view backs both the Student-facing progress bar on a learning path's landing page and the Analytics dashboards (§2.17). Computing it on read (with the short-TTL Redis caching pattern from §5.4) rather than maintaining a separate progress-state table is the correct MVP trade-off because the source data (`investigation_sessions`, `scores`) is already the single source of truth — a separate progress table would require careful invalidation logic for no accuracy benefit at this data volume.

13.3 Achievements

`achievements` (§6.21) are awarded by a background job (`reporting` queue, §5.9, or a dedicated lightweight `achievements` job triggered on session submission) evaluating each active achievement's `criteria` JSONB against the submitting user's updated history — e.g., `{"type": "category_mastery", "category": "ransomware", "min_sessions": 5, "min_avg_score": 90}`. Achievement criteria are evaluated only on submission events, not via a continuous polling scan, so this stays cheap at scale (§3.3). Earned achievements are recorded in `user_achievements` and surfaced on the Student's profile and, optionally, on the leaderboard (§13.5) as a visible badge.

13.4 Certificates

Covered structurally in §6.18/§2.18. Issuance flow: on learning-path completion meeting `pass_threshold_percent`, a `reporting` queue job renders a PDF (server-side HTML-to-PDF rendering, using the same rendering approach as incident report export, §2.19, for one consistent PDF pipeline rather than two) embedding the learner's name, path title, completion date, and a QR code/URL to the public verification page (`/verify/{certificate_uuid}`, §2.18), stores it in the `certificates` object storage bucket (§5.11), and inserts the `certificates` row with a `score_snapshot` so the certificate's displayed data remains stable and renderable even after the underlying session telemetry is archived per the 90-day retention policy (§6.23).

13.5 Leaderboards

`leaderboard_entries` (§6.21) are recomputed by the scheduled rollup job (§5.12) per period (`weekly/monthly/all_time`), ranking users by a points formula combining scenario completions, average score, and a difficulty multiplier (an `expert`-tier completion is worth more points than a `beginner`-tier completion at the same score, so the leaderboard rewards tackling harder content rather than farming easy scenarios). Leaderboards are scoped three ways in the UI: global, within-org (institutional/enterprise customers, §2.16), and within-cohort (Instructor Mode, §2.15) — all three read from the same underlying `leaderboard_entries` table filtered by `org_id`/cohort membership, not three separate tables. Participation is opt-in at the account level (a `users` preference, not modeled as a separate column in §6.3's table but implied as part of the account-settings surface, §17) since some institutional buyers' learners may be uncomfortable with public ranking.

13.6 Learning Paths and Career Tracks

At MVP launch, two `career_track` values are seeded: **SOC Analyst Career Track** (breadth-first: one learning path touching each category in §1.8 at `beginner` difficulty, then a second pass at `intermediate`) and **Blue Team Career Track** (depth-first: multiple learning paths per category — e.g., three consecutive ransomware-focused paths at increasing difficulty — aimed at learners specializing rather than surveying). Both tracks are simply curated `courses/learning_paths` content, not distinct code paths — the "career track" concept is a content-organization and marketing construct (used in course-catalog browsing/filtering, §17) layered on top of the same scenario/learning-path data model, keeping the platform capable of adding new tracks purely through content authoring (§12.1) with no engineering work required.

13.7 Instructor-Facing Learning Platform Features

Cohort scenario assignment (§2.15, `cohort_scenario_assignments`, §6.6) can target either an individual `attack_scenarios` row or an entire `learning_paths` row (assigning the whole path assigns every scenario in it with the same due date, recorded as one assignment row referencing the path, resolved to individual scenario obligations by the gradebook view rather than expanded into N rows at assignment time — keeping the assignment model simple and the due-date edit a single-row update).

14. AI Features

14.1 Scope and Positioning

Every capability in this section is a **Phase 3** initiative (§1.7) — none is required for MVP launch, and none is permitted to sit in the critical path of core scenario generation, alerting, or scoring (§7.6 already establishes that core telemetry generation has no runtime LLM dependency, for cost/latency/reproducibility reasons that still apply). AI features here are additive assistive layers on top of the deterministic core, each designed so that **disabling the AI feature entirely still leaves a fully functional product** — this is a hard architectural constraint, not a nice-to-have, because it keeps SOCVerse's core value proposition (a reliable, reproducible investigation simulator) independent of third-party LLM API availability, pricing changes, or quality regressions.

All AI features are built behind a single internal `AIGatewayService` (§18) abstraction so the underlying model provider is swappable without touching feature code, and every AI-generated response is logged (prompt, response, model/version, cost) for quality monitoring and for the abuse/cost controls in §14.7.

14.2 AI Investigation Assistant

A chat-style panel available inside an active incident (§2.3), scoped strictly to that incident's own data: the assistant's context window is constructed server-side from the incident's linked alerts, evidence collection, notes, and timeline — **never** from the scenario's `ground_truth_definition**` or any `is_ground_truth_*`-flagged field (§12.3's serialization-layer stripping applies identically to whatever context is assembled for an AI prompt, since a prompt leak is just as damaging to assessment validity as an API response leak). The assistant can answer questions like "what devices has this identity signed into?" (answered by calling the same internal read APIs as the UI, then summarizing — i.e., retrieval-augmented, not generated from parametric knowledge) and can be asked to explain a concept (e.g., "what is impossible travel and why does it matter here") using general security knowledge, clearly distinguished in the UI from anything data-grounded. It explicitly refuses to state a verdict or name the "correct" technique — it is a research aid, not an answer key; this behavioral boundary is enforced via a system prompt plus a post-response filter that blocks responses containing scenario-specific ground-truth identifiers.

14.3 AI Hint System

An optional enhancement to the authored hint system (§12.5): rather than only surfacing pre-written hint text, an AI layer can *rephrase or contextualize* an authored hint for the specific Student's current investigative state (e.g., referencing an alert they've already dismissed) — but the underlying hint content and its `unlock_cost_percent` penalty remain fully authored and deterministic (§12.5); the AI layer only affects presentation, never which hint is available or its cost, so scoring integrity is unaffected regardless of whether this feature is enabled.

14.4 Natural Language Search

A convenience layer in front of the existing structured Search feature (§2.8): a Student's freetext question ("show me failed sign-ins from Russia last night") is translated server-side into the existing `field=value` query structure and executed through the same Search service (§18.3) — the NL layer produces a *query*, never a

result set itself, and the resulting structured query is always shown to the Student alongside the results (an editable "here's what I searched for" chip) so they build the same query-construction literacy the platform is trying to teach, rather than the AI silently doing the investigation for them.

14.5 AI Report Writer

Assists a Student in drafting their final incident summary (§2.3, §2.11) by generating a **first draft** from their own Evidence Collection, Notes, and Timeline entries — again strictly retrieval-augmented from the Student's own curated data, never from ground truth — which the Student must review and edit before submission. The submission form always requires the summary text differ meaningfully from the AI draft by a minimum edit-distance threshold, or requires an explicit "I reviewed and confirm this reflects my own analysis" checkbox, specifically to discourage pure copy-through-without-review, since the writing/reasoning process itself has pedagogical value the platform does not want to let learners skip entirely. This same underlying capability, applied to **scenario authoring** rather than Student reporting, is the "author-time AI assistance" mentioned in §7.6 — helping a human scenario author draft email body text, decoy document content, or phrasing variations for the template library (§7.3), strictly offline/at authoring time, with all output reviewed and approved by a human author before entering the template library.

14.6 AI Feedback

A Phase 3 enhancement to §6.20's `instructor_feedback`: for self-directed (non-cohort) Students who have no human instructor, an optional AI-generated feedback pass can supplement the automated `scores.rubric_breakdown` with prose-style, encouraging, specific commentary ("You correctly identified the phishing vector but missed the lateral movement into the finance server — check the sign-in timeline for DEV-FIN-03 around 14:00") generated from the same `rubric_breakdown` data already computed deterministically by the Scoring Engine (§12.4) — the AI's role is exclusively to narrate an already-computed, deterministic result in natural language, never to compute or adjust the score itself, keeping grading fully deterministic and auditable regardless of this feature's availability.

14.7 SOC Copilot (Longer-Term Vision)

The longer-term, most ambitious AI capability: a unified assistant spanning the whole active session (not just one incident) that can proactively surface patterns across alerts a Student hasn't yet connected — explicitly framed as a **higher-tier or later-course-unlock feature**, since over-relying on it too early would undercut the platform's core assessment value. Positioned as the natural product bridge toward the "certification exam attempt" commercial product in §1.7 Phase 3: an exam-mode session would disable the Copilot (and all other AI assistance) entirely, while standard practice-mode sessions may enable it, giving instructors/orgs (§2.16) a per-cohort or per-assignment toggle controlling AI-assistance availability — modeled as a boolean on `cohort_scenario_assignments` (§6.6) and a default account-level preference for self-directed learners.

14.8 Cost and Abuse Controls

Every AI feature call passes through `AIGatewayService`'s per-user and per-org rate limits and a hard monthly token/cost budget per subscription tier (§20), with graceful degradation (a clear "AI assistance temporarily unavailable, here are the standard tools" message, never a hard error) if a budget is exceeded — consistent with §3.8's reliability principle that a dependency failure must degrade gracefully rather than

break the core product. Given the cost sensitivity established in \$1.7/\$20, AI features are explicitly scoped as a **post-revenue** investment: they are designed here so the architecture is ready, not built or budgeted for at MVP launch.

15. Security Architecture

15.1 Authentication

- Passwords: argon2id (memory-hard, GPU-resistant), tuned parameters reviewed annually against current OWASP guidance; never MD5/SHA1/bcrypt-only.
- Minimum password policy: length-based (12+ characters) rather than arbitrary complexity rules, per current NIST 800-63B guidance, checked against a common-password blocklist at signup/change time.
- Email verification required before a self-serve account can start a scored (leaderboard/certificate-eligible) session; unverified accounts can still explore free-tier content to avoid a signup-friction cliff.
- Account lockout: progressive delay + eventual temporary lockout after repeated failed attempts (Redis-backed counter keyed by account + IP, distinct from the general API rate limiter in §15.5), with an `audit_logs login_failed` entry per attempt (§6.22) for anomaly review.
- MFA: TOTP-based optional MFA for individual accounts, **mandatory** for Org Admin and Platform Admin roles given their access breadth (§15.2) — enforced at login, not just offered.
- SSO (SAML 2.0 / OIDC): Phase 2 addition for institutional buyers (§5.5), terminating into the same internal JWT issuance path as password auth so downstream authorization logic is identical regardless of auth method.

15.2 Authorization (RBAC)

Four platform roles (`users.role`, §6.3): `student`, `instructor`, `org_admin`, `platform_admin`.

Authorization is enforced in two layers, both required (defense in depth — neither is trusted alone):

26. **Coarse, route-level (API Gateway, §5.2):** a static role→route matrix rejects a request before it reaches any business logic if the role can never legitimately call that route (e.g., `student` can never call any `/admin/*` route).
27. **Fine, resource-level (owning domain service, §4.3 step 4):** does *this specific* caller own or have a legitimate relationship to *this specific* resource — e.g., does `sessionId` belong to `user_id`; is `instructor_id` actually the owner of the cohort containing this Student; is `org_id` on the caller's token the same `org_id` as the target resource. This check happens on every read and write, not only writes, since unauthorized read access to another Student's session would itself leak ground-truth-adjacent information and violate the assessment-integrity goals in §12.3.

Role capability summary: `student` — own sessions/scores/certificates only. `instructor` — own cohorts, enrolled Students' sessions within those cohorts (read + feedback, never edit the Student's own submission), scenario authoring (Phase 2). `org_admin` — own org's users/cohorts/billing/SSO config, no cross-org access, no access to specific Student investigation content beyond aggregate analytics (§2.17) unless also holding an instructor relationship. `platform_admin` — full platform access, gated additionally by mandatory MFA (§15.1) and full audit logging of every admin action including read-level "support" access (§15.9).

15.3 Encryption

- **In transit:** TLS 1.2+ enforced on every public endpoint (HTTP requests to port 80 receive a redirect only, no data is ever served over plaintext HTTP); internal service-to-service traffic at Scale (§19.4) runs over the cluster's private network with TLS between mesh-connected services where the deployment platform supports it, and is never exposed to the public internet regardless (§4.5).
- **At rest:** the database volume and object storage are encrypted at rest using the host/provider's disk or bucket-level encryption (LUKS on the MVP VPS; provider-managed encryption at Scale) — application-layer field encryption is *not* used for the synthetic telemetry tables (there is no real-world sensitive content there to protect beyond standard disk encryption) but **is** used for `organizations.sso_config` secret-bearing fields and any stored payment-related reference tokens (§20 — SOCVerse never stores raw card data itself; billing integrates with a PCI-compliant processor, §15.8).
- Backups (§3.11) inherit the same at-rest encryption and are additionally encrypted with a backup-specific key before leaving the primary environment.

15.4 Secrets Management

Covered operationally in §5.17. Security-specific requirements: secrets are never logged (a log-scrubbing middleware redacts known secret-shaped fields — `password`, `token`, `secret`, `authorization` — from any structured log line before it is written, as a defense-in-depth backstop against a developer accidentally logging a full request object); the JWT signing key is rotated on a routine schedule and immediately on any suspected compromise, with a brief dual-key acceptance window (old key still validates existing unexpired tokens; new key signs everything new) to avoid mass session invalidation on rotation.

15.5 Rate Limiting

Redis-backed sliding-window limiter at the API Gateway (§5.2), tiered: unauthenticated endpoints (login, signup, password reset) limited per-IP tightly (e.g., 10/minute) given their abuse potential (credential stuffing, account enumeration); authenticated endpoints limited per-user at a generous level tuned to not interfere with normal investigative click-through speed, with a stricter limit specifically on session-start/telemetry-generation-triggering endpoints (§7.7) since each is a real compute cost; a separate, much stricter limiter applies to AI feature endpoints (§14.8) given their per-call cost. Exceeding a limit returns 429 with a `Retry-After` header; sustained abuse from one identity escalates to a temporary account flag reviewed by Platform Admin tooling (§2.16).

15.6 API Security

- Every request body is validated against its OpenAPI-derived schema (§5.2, §16) before reaching handler code — type, required-field, and length constraints rejected at the edge.
- Object storage access is exclusively via short-lived signed URLs (§5.11); no bucket is ever public.
- CORS is restricted to the platform's own frontend origin(s); no wildcard origin in production.
- Sensitive response fields (§9.9, §10.12, §11.10, §12.3) are stripped at a shared serialization layer, not per-endpoint — a new endpoint inherits safety by construction rather than requiring the author to remember to strip fields.

- CSRF is not applicable to the primary API surface (Bearer-token JWT auth, not cookie-session auth, §5.7), but any cookie-based flow that is introduced (e.g., a "remember me" refresh-token cookie option) uses `SameSite=Strict/Lax` plus standard double-submit CSRF protection.
- Rendered email HTML (§11.10) is sanitized server-side before ever reaching the client.

15.7 Session Management

Detailed token mechanics in §5.7. Security-relevant behaviors: refresh-token rotation with reuse detection (a previously-rotated-away token being presented again immediately revokes the entire token family and forces re-authentication, since it indicates token theft, §6.19); `session_version` bump on password change/MFA change/explicit "log out everywhere" instantly invalidates all previously-issued access tokens without requiring a server-side blocklist of every token (§5.7); idle-session frontend timeout with a re-authentication prompt for Org/Platform Admin sessions specifically, given their access sensitivity.

15.8 Payment Security

SOCVerse never handles, stores, or transmits raw payment card data — all billing/subscription payment collection is delegated to a PCI-DSS-compliant third-party payment processor's hosted checkout/tokenization flow (§20); the platform stores only the processor-issued customer/subscription reference tokens (`organizations.billing_customer_id`, §6.4, and an equivalent field on `users` for individual subscriptions), keeping SOCVerse itself out of PCI-DSS scope entirely — a deliberate, cost- and risk-reducing architectural choice appropriate for a bootstrapped team (§20).

15.9 Audit Logging

Full schema in §6.22. Security-specific requirement beyond what's stated there: **admin impersonation** (a Platform Admin support tool allowing "view as this user" for troubleshooting) is logged with its own distinct `audit_logs.action = 'admin_impersonation_started'/'ended'` pair including the impersonated `target_id`, is time-boxed (auto-expires after a short window, requiring re-initiation rather than a standing session), is visually indicated in the UI at all times during an impersonated session (a persistent banner, never a silent capability), and — critically — impersonation sessions are read-only by default; any state-mutating action taken while impersonating requires a distinct, separately-logged explicit confirmation step, since silent support-driven mutation of a Student's graded work would undermine the assessment integrity the whole platform depends on.

15.10 Input Validation

Beyond §15.6's schema-level validation: all user-authored free text (Analyst Notes §2.13, incident summaries §2.3, instructor feedback §6.20) is treated as untrusted on output (rendered with framework-default HTML-escaping, never `dangerouslySetInnerHTML/innerHTML`-equivalent raw injection, §17) even though it is never executed server-side — this is standard stored-XSS defense applied consistently regardless of the (low) likelihood of a malicious Student in an education product, because the cost of applying it uniformly is near-zero and the cost of an XSS incident (especially one that could reach an Instructor or Admin viewing Student-authored content) is not.

15.11 Secure Coding Practices

- Dependency scanning (automated, in CI, §5.1) against known-vulnerability databases for both frontend and backend dependency trees, blocking merges on newly-introduced critical vulnerabilities.
- Static analysis/linting includes a security-focused rule set (no `eval`, no unsanitized template construction, no hardcoded secrets — the last backed by a pre-commit secret-scanning hook, §5.17) as a required CI gate, not an optional warning.
- No real malicious code, functional exploit payloads, or live-callback infrastructure is ever present anywhere in the platform's synthetic content (§7.6, §11.5) — every "malicious" artifact a Student encounters is inert metadata, by explicit content-authoring policy enforced in the scenario validation pipeline (§12.6).
- Principle of least privilege applied to infrastructure credentials: the application's database role has no DROP/ALTER grants in production (migrations run under a separate, more privileged role invoked only by the CI/CD migration step, §6.24); the `audit_logs` table specifically revokes UPDATE/DELETE from the application role (§6.22).

15.12 Data Subject Rights / Account Deletion

An account-deletion request (self-serve or support-initiated) triggers a background job that anonymizes/removes personal data from `users` (email, display name) while preserving the row as a tombstone (soft delete, §6.1) so that referential integrity for aggregate analytics (§2.17) and cohort/institutional records (which may have independent retention obligations, e.g., an institution's own academic record requirements) is not broken; all directly-identifying content the user authored (Analyst Notes, incident summaries containing their own writing) is deleted outright rather than anonymized-and-kept, since anonymized free text can still be re-identifying. This flow is documented in a runbook and is a Platform Admin-triggered action, logged in `audit_logs`, not a fully self-service instant-delete button at MVP, given the cross-table cleanup involved — full self-service deletion is a Scale-phase enhancement once volume justifies automating it further.

16. API Specification

16.1 Conventions

- Base path: `/api/v1`. All requests/responses are `application/json` (UTF-8). All endpoints except `/auth/*` and `/public/*` require `Authorization: Bearer <access_token>` (§5.7).
- Every response includes an `X-Correlation-Id` header (§5.2). Every list endpoint supports `?page=&pageSize=` (default `pageSize=25`, max `100`) and returns a standard envelope: `{ "data": [...], "meta": { "page", "pageSize", "totalCount" } }`.
- Errors follow a standard shape: `{ "error": { "code": "STRING_CODE", "message": "human readable", "correlationId": "uuid" } }` with the appropriate HTTP status (`400` validation, `401` unauthenticated, `403` unauthorized, `404` not found, `409` conflict, `429` rate limited, `500` unexpected).
- A machine-readable OpenAPI 3.1 document is the canonical, generated-from-code source of truth in the actual repository; this section is the human-readable reference derived from it and takes precedence only where the two might drift during initial build-out.

16.2 Auth Service (`/api/v1/auth`)

`POST /auth/signup`

Request: `{ "email": "a@b.com", "password": "correct horse battery staple", "displayName": "A. Student" }`

Response `201`: `{ "userId": "uuid", "emailVerificationRequired": true }`

`POST /auth/login`

Request: `{ "email": "a@b.com", "password": "..." }`

Response `200`: `{ "accessToken": "jwt", "refreshToken": "opaque", "expiresIn": 900, "user": { "id", "displayName", "role" } }`

Response `401`: standard error envelope, code: `"INVALID_CREDENTIALS"`.

`POST /auth/refresh`

Request: `{ "refreshToken": "opaque" }` → Response `200`: new `{ accessToken, refreshToken, expiresIn }` (rotated, §5.7).

`POST /auth/logout` — revokes the presented refresh token. `POST /auth/logout-all` — bumps `session_version` (§5.7), revokes all refresh tokens for the user.

`POST /auth/mfa/verify` — `{ "mfaChallengeId", "code" }` → completes login for MFA-enrolled accounts (§15.1).

16.3 Scenario Catalog Service (`/api/v1/scenarios`)

``GET /scenarios`` — filters: `?category=&difficulty=&careerTrack=`. Response 200:

```
{ "data": [ { "id", "slug", "title", "summary", "category", "difficulty", "estimatedMinutes" } ], "meta": { ... } }
```

(never includes `ground_truth_definition` or any internal version metadata beyond `id`, §12.3).

``GET /scenarios/{slug}`` — full learner-facing detail for the catalog page (§17.5).

16.4 Session Service (`/api/v1/sessions``)

``POST /sessions``

Request: { `"scenarioId": "uuid", "cohortAssignmentId": "uuid|null"` }

Response 201: { `"sessionId": "uuid", "status": "provisioning"` } — provisioning if a synchronous generation path was taken (§7.7); client polls `GET /sessions/{id}` or listens on the WebSocket channel for a `session.ready` event.

``GET /sessions/{id}`` → { `"id", "scenarioId", "status", "startedAt", "expiresAt", "currentScenarioTime"` }.

``POST /sessions/{id}/submit``

Request: { `"incidentIds": ["uuid", ...]"` }

Response 202: { `"scoringStatus": "queued"` } — scoring runs async (§12.4); client polls `GET /sessions/{id}/score` or listens for `score.ready`.

``GET /sessions/{id}/score`` → full `scores` row shape including `rubricBreakdown` (§6.14), 404 if not yet scored.

16.5 Alert Service (`/api/v1/sessions/{sessionId}/alerts``)

``GET /alerts`` — filters: `?severity=&status=&entityType=`. Response item shape:

```
{ "id", "title", "description", "severity", "status", "primaryEntityType", "primaryEntityId", "mitreTechnique": { "id", "techniqueId", "name" }, "dedupCount", "firstSeenAt", "lastSeenAt" }
```

``GET /alerts/{id}/evidence`` → { `"alertId", "evidence": [ { "eventTable", "eventId", "occurredAt", "summary" } ]` } — resolved from `alert_evidence_refs` (§6.15), each `summary` a redacted, Student-safe rendering of the underlying event.

``PATCH /alerts/{id}``

Request: { `"status": "dismissed", "dismissalReason": "Expected admin RDP session per change ticket."` } — `dismissalReason` required (400 if missing) when `status = "dismissed"` (§2.1, §8.6).

16.6 Incident Service (`/api/v1/sessions/{sessionId}/incidents``)

``POST /incidents`` → { "title": "Suspicious sign-in + malware execution on FIN-WKS-04" } → 201 with new incident.

``POST /incidents/{id}/alerts`` → { "alertIds": ["uuid"] } — links alerts (§2.2).

``PATCH /incidents/{id}`` — status transitions (§2.3): { "status": "investigating" }.

``POST /incidents/{id}/close``

Request:

```
{ "verdict": "true_positive", "summary": "...", "mitreTechniqueIds": ["uuid", "uuid"] }
```

Response 200: closed incident; 409 if evidenceCollection count is below the scenario's min_evidence_items (§12.1) with code: "INSUFFICIENT_EVIDENCE".

16.7 Evidence, Notes, and Timeline

(``/api/v1/sessions/{sessionId}/incidents/{incidentId}``)

``POST /evidence`` → { "eventTable": "process_events", "eventId": "uuid", "justification": "Unsigned binary spawned from Outlook.", "mitreTechniqueId": "uuid|null" }.

``GET /evidence`` / ``DELETE /evidence/{id}``.

``POST /notes`` → { "body": "..." }; ``GET /notes``.

``GET /timeline`` → merged, chronologically-sorted view across every evidence-collection item's source event plus manually-added items, each entity-tagged for swimlane rendering (§2.9).

16.8 Investigation Actions (``/api/v1/sessions/{sessionId}/actions``)

``POST /actions``

Request: { "actionType": "isolate_device", "targetType": "device", "targetId": "uuid", "incidentId": "uuid|null" }

Response 200: { "actionId", "occurredAt" } — also mutates the target entity's state where applicable (e.g., devices.isolation_status, §10.11) within the same transaction.

16.9 Identity Portal (``/api/v1/sessions/{sessionId}/identities``)

``GET /identities`` — filters: ?riskLevel=&department=&mfaStatus=&hasRiskySignins=true.

``GET /identities/{id}`` → full profile (§9.3): overview, devices, groups, conditionalAccess, riskLevel, mfaStatus.

``GET /identities/{id}/signins`` — filters: ?result=&from=&to=&riskyOnly=true; each item includes computed distanceFromPreviousKm/impliedTravelSpeedKmh when a previous sign-in exists (§9.6).

16.10 Device Portal (``/api/v1/sessions/{sessionId}/devices``)

``GET /devices`` / ``GET /devices/{id}`` → overview + `installedSoftware`, `services`, `startupEntries` (§10.2).

``GET /devices/{id}/process-tree`` → nested tree structure rooted at each top-level process (§10.3), `?commandLineContains=` filter flattens-and-highlights matches without pruning ancestor/descendant context.

``GET /devices/{id}/network``, ``GET /devices/{id}/files``, ``GET /devices/{id}/registry``, ``GET /devices/{id}/usb``, ``GET /devices/{id}/timeline`` (§10.8, consolidated cross-artifact feed).

16.11 Email Portal (``/api/v1/sessions/{sessionId}/emails``)

``GET /emails`` — filters per §11.2. ``GET /emails/{id}`` → full message incl. `headersRaw`, `authResults: { spf, dkim, dmarc }`, `attachments[]`, `urls[]`.

``GET /emails/{id}/similar`` → campaign correlation results (§11.8).

16.12 Search and Threat Intel (``/api/v1/sessions/{sessionId}``)

``POST /search`` → `{ "filters": [ { "field": "sourceIp", "value": "45.x.x.x" } ], "freetext": "optional" }` → unified cross-table result list, each item entity-tagged with one-click `addToTimeline/pinEvidence` action references (§2.8).

``GET /threat-intel?type=hash&value=...`` → `{ "value", "type", "reputation", "actorAttribution", "context" }`, 404 (rendered client-side as "Unknown") if not present in the scenario's `threat_intel_indicators` (§2.7).

16.13 Instructor Service (``/api/v1/instructor``)

``POST /cohorts``, ``GET /cohorts/{id}/roster``, ``POST /cohorts/{id}/assignments`` → `{ "scenarioId|learningPathId", "dueAt", "attemptLimit", "aiAssistanceEnabled": true }` (§14.7).

``GET /cohorts/{id}/review-queue`` → sessions awaiting review, each with automated scores alongside a link to the full evidence trail.

``POST /incidents/{id}/instructor-feedback`` → `{ "rubricOverrides": {...}, "comment": "...", "reopenSession": false }` (§6.20).

``GET /cohorts/{id}/gradebook.csv`` → CSV export (§2.15).

16.14 Admin Service (``/api/v1/admin``)

``GET /admin/scenarios``, ``POST /admin/scenarios/{id}/versions``, ``POST /admin/scenarios/{id}/publish`` (runs the validation pipeline, §12.6; 422 with itemized validation failures on failure).

``GET /admin/orgs/{id}/usage`` (§2.17). ``POST /admin/impersonate`` → `{ "targetUserId" }` → short-lived, clearly-flagged impersonation token (§15.9).

``GET /admin/audit-logs`` — filters: `?actorUserId=&action=&from=&to=` (§6.22).

16.15 Learning Platform Service (`/api/v1/learning`)

`GET /courses`, `GET /paths/{id}` (with computed progress, §13.2), `GET /leaderboard?period=weekly&scope=org` (§13.5), `GET /certificates/mine`, `GET /public/verify/{certificateId}` (unauthenticated, §2.18).

16.16 WebSocket Channel

wss://.../ws?sessionId={id} (auth via first-message frame { "type": "auth", "accessToken": "..."}, §5.10). Server→client message types:

```
{ "type": "alert.new", "payload": { ...alert shape from §16.5 } }
{ "type": "alert.updated", "payload": { "id", "status" } }
{ "type": "incident.status_changed", "payload": { "id", "status" } }
{ "type": "session.time_advanced", "payload": { "currentScenarioTime" } }
```

Every message type has a REST fallback (§5.10) so a missed/dropped socket message never leaves client state permanently stale (§17.9).

17. Frontend Architecture

17.1 Application Shells

One codebase, one build, four role-scoped shells selected post-authentication by `users.role` (§6.3), each with its own top-level layout and navigation but sharing the full component library (§17.4) and API client (§17.6):

- **Student Shell** — the primary product surface: scenario catalog, active session workspace (Alert Dashboard, Incident Queue, the four investigation portals, Search, Timeline), progress/leaderboard/certificates.
- **Instructor Shell** — cohort management, review queue, gradebook, scenario authoring entry point (§2.15).
- **Org Admin Shell** — seats, SSO config, billing, org-level analytics (§2.16).
- **Platform Admin Shell** — scenario library management, global user/support tools, platform analytics, audit log viewer.

Route guards check role at the router level (redirecting to the correct shell's home if a user lands on a route outside their role) as a UX convenience only — the actual authorization boundary is enforced server-side (§15.2) and the frontend never trusts a client-side check as a security control.

17.2 Routing

File-system-based or centrally-declared routing (implementation detail, §5.1) under top-level prefixes matching the shells: `/app/*` (Student), `/instructor/*`, `/org/*`, `/admin/*`, plus unauthenticated routes `/login`, `/signup`, `/verify/{certificateId}` (public certificate verification, §2.18), and marketing/catalog pages that may be server-rendered or statically generated separately from the authenticated SPA for SEO purposes (course/scenario catalog pages benefit from being crawlable, §1.3's content-marketing acquisition motion). The active-session workspace uses nested routing under `/app/sessions/{sessionId}/...` with each investigation portal, the dashboard, and incident detail as sibling routes sharing a persistent session-level layout (top nav showing session timer/status, always-visible Global Timeline access, §2.9).

17.3 State Management

Three distinct state categories, each handled by the tool suited to it rather than one global store for everything:

28. **Server state** (alerts, incidents, entity profiles, scores — anything from §16) — managed by a query/cache library (e.g., TanStack Query or equivalent) providing request deduplication, background refetch, and cache invalidation keyed by the same resource identifiers as the API, with WebSocket events (§16.16) triggering targeted cache invalidation/patching (an `alert.new` message pushes directly into the alerts list cache rather than forcing a full refetch, for responsiveness).

29. **Client/UI state** (open panel, selected filter, draft note text before autosave) — local component state or a lightweight store, deliberately not persisted to any global app-wide state container, since this state is cheap to lose on navigation and coupling it to the server-state layer would only add complexity.
30. **Auth/session state** (current user, role, access token) — a small dedicated auth context/store, the only genuinely global piece of client state, responsible for attaching the bearer token to every API client request (§17.6) and reacting to 401s by attempting a silent refresh (§5.7) before forcing re-login.

17.4 Component Library and Design System

A shared component library (buttons, tables, filter builders, severity badges, entity-type icons, the process-tree renderer, the timeline swimlane renderer) is built once and consumed by all four shells — the severity badge, MITRE technique chip, and entity-profile-link components in particular appear in nearly every screen (Alert Dashboard, Search results, Timeline, all three investigation portals) and are the components most worth investing shared-design-system rigor in, since visual/interaction consistency across those repeated elements is a large part of what makes the product feel like a coherent, professional-grade console rather than a collection of separately-built pages (§1.2, §1.6).

17.5 Key Pages and Navigation Flow

- **Catalog → Scenario Detail → Start Session** (/app/catalog → /app/catalog/{slug} → POST /sessions → redirect to /app/sessions/{id}/dashboard).
- **Session Dashboard** (/app/sessions/{id}/dashboard) — Alert Dashboard (§2.1) as the landing view within an active session, with persistent left-nav to Incident Queue, Identities, Devices, Emails, Search, Timeline, and a session status/timer indicator.
- **Alert Detail** (modal or /app/sessions/{id}/alerts/{alertId}) — evidence drill-down, promote-to-incident action.
- **Incident Workspace** (/app/sessions/{id}/incidents/{incidentId}) — the densest screen in the product: linked alerts, entity chips, embedded Timeline scoped to the incident, Evidence Collection panel, Analyst Notes panel, and the closure/verdict form — laid out as a three-column workspace (context/notes | primary evidence-timeline | linked-entities-and-actions) on desktop, collapsing to a tabbed single-column layout on narrower viewports (§17.7).
- **Identity/Device/Email Portal** (/app/sessions/{id}/identities, /devices, /emails) — list → profile detail, each profile deep-linkable and cross-linkable from anywhere else in the product (e.g., an alert's primary entity chip links directly to that entity's profile).
- **Results Screen** (/app/sessions/{id}/results) — post-submission score breakdown (§2.12), missed-evidence callouts, hint-usage summary, "review full session" (read-only replay of the completed session using `investigation_actions`, §6.13) and "try again"/"next in path" calls to action.
- **Instructor Review Queue → Session Review** — side-by-side automated-score view and full evidence-trail replay, with the feedback form (§16.13) docked alongside.

17.6 API Client Layer

A single generated (from the OpenAPI spec, §16.1) typed API client is the only code path allowed to call the backend — no ad hoc `fetch` calls scattered through components — providing compile-time safety against the API contract drifting out from under the frontend, automatic bearer-token attachment and refresh-on-401 retry (§17.3), and centralized error-envelope handling (mapping the standard error shape from §16.1 into user-facing toast/inline messages consistently across the app).

17.7 Responsive Behavior

The Student investigation workspace is designed **desktop-first** (multi-panel layouts, process trees, and timelines genuinely need screen real estate to be usable, and this mirrors the real professional tooling context being taught, §1.6) but must remain fully usable down to a tablet-width breakpoint (collapsing multi-column workspaces to tabs, as noted in §17.5) since institutional/classroom usage (§1.3) cannot assume every learner has a large monitor. Below tablet width, the product shows a clear "best experienced on a larger screen" notice on session-workspace routes rather than attempting a fully-featured phone layout — catalog browsing, progress/leaderboard/certificate viewing, and marketing pages remain fully mobile-responsive, since those are plausible phone use cases (§1.4) while active investigation is not.

17.8 Accessibility

WCAG 2.1 AA is the baseline target: full keyboard operability for every investigative action (evidence pinning, timeline adding, alert status changes all reachable and actionable without a mouse), semantic HTML and ARIA labeling for the custom process-tree/timeline/severity-badge components specifically (since these are the components most likely to be built as non-standard widgets and therefore most likely to be missed by default), sufficient color contrast including for severity-badge color coding (never color-alone — severity is always paired with a text label/icon, so colorblind users are not dependent on hue distinction), and full support for browser zoom/text-resizing without layout breakage on the dense incident workspace.

17.9 Real-Time and Offline-Degradation Behavior

Every screen that consumes WebSocket push updates (§16.16) is built to be **fully correct from a REST GET alone** — the socket is purely a latency optimization, never a data-integrity dependency (§5.10, §3.8). If the WebSocket connection drops, the client falls back to a modest-interval poll (e.g., every 15s) of the affected list endpoints (alerts, incident status) with a small, non-intrusive "reconnecting" indicator, and resumes push-based updates transparently on reconnection — a Student should never be able to tell, other than a brief indicator, that the underlying transport degraded.

17.10 Performance Budget

Code-splitting by shell and by route (the Instructor/Org/Platform Admin shells are never downloaded by a Student session, and vice versa) keeps the Student Shell's initial bundle focused on catalog + auth + dashboard, with the heavier investigation-portal and process-tree/timeline rendering code lazy-loaded on first entry into an active session — directly supporting the §3.1 Time-to-Interactive target by ensuring the first meaningful paint never waits on code the current screen doesn't need.

18. Backend Architecture

18.1 Service Inventory

Restating and detailing the domain services first introduced in §4.2, each a distinct code module (physically separate service at Scale, §4.6):

Service	Owns tables (§6)	Responsibilities
Identity/Auth Service	users, organizations, cohorts, cohort_enrollments, refresh_tokens	Signup/login/SSO, RBAC data, org/cohort membership
Scenario & Content Service	attack_scenarios, scenario_versions, scenario_techniques, mitre_techniques, cohort_scenario_assignments	Catalog, authoring, versioning, publish validation (§12.6)
Session Service	investigation_sessions	Session lifecycle, orchestrates generation (§7) and scoring (§12.4) triggers
Telemetry Generator (worker)	identities, devices, all §6.12 event tables	Synthetic world/event generation (§7)
Alert Engine (worker)	alerts, alert_evidence_refs, detection_rules, threat_intel_indicators	Detection/correlation (§8)
Investigation Service	incidents, incident_alerts, incident_techniques, evidence_collection, analyst_notes, investigation_actions	Case management, evidence, notes, actions (§2.2–§2.10, §2.20)
Scoring Engine (worker)	scores, score_history	Automated grading (§12.4)
Identity/Device/Email Portal Services	(read-mostly views over §6.10–§6.12)	Investigation-surface read APIs (§9–§11)
Learning Platform Service	courses, learning_paths, achievements, certificates, leaderboard_entries	§13
Instructor Service	instructor_feedback	§2.15, §16.13
Admin Service	(cross-cutting reads + audit_logs writes)	§2.16
Notification/Realtime Module	(no owned tables; Redis pub/sub, §5.10)	WebSocket fan-out
Reporting Worker	(reads across domains, writes to object storage)	PDF/CSV export (§2.19)
AI Gateway Service (Phase 3)	(no owned tables; logs prompt/response metadata)	§14

Each service exposes its functionality only via its public interface (in-process function calls at MVP, HTTP+internal-auth at Scale, §4.6) — **no service queries another service's tables directly**, even though they may be physically co-located in one Postgres instance at MVP; this is enforced by code-organization convention (each service's repository layer is package-private to that service) and checked by a lint rule scanning for cross-package table-client imports in CI.

18.2 Identity/Auth Service — Internal Workflow Example

POST /auth/login (§16.2) flow: validate request shape → look up `users` by email (constant-time comparison path not required here since email isn't a secret, but the subsequent password check uses argon2id's built-in constant-time verify) → on hash mismatch, increment the Redis-backed failed-attempt counter (§15.1) and write an `audit_logs login_failed` row → on success, check `mfa_status`; if MFA-enrolled, issue a short-lived MFA challenge token and return 202 pending MFA rather than tokens; if not, issue access+refresh tokens (§5.7), write `audit_logs login` row, update `last_login_at`, return 200. Every branch — success, bad password, MFA-pending, locked-out — is a distinct, tested path, since auth is the highest-consequence code in the service.

18.3 Cross-Cutting: The Serialization/DTO Layer

Referenced repeatedly in §9.9/§10.12/§11.10/§12.3/§14.2/§15.6: every domain service that touches a table with a ground-truth or internally-sensitive column defines its Student-facing response DTOs as an explicit allow-list of fields (never "return the row minus a blocklist"), constructed by a shared `toStudentDTO()`-style mapping function per entity type, used uniformly by every endpoint that returns that entity type (list, detail, search-result, timeline-item, AI-assistant-context-builder, §14.2). A distinct, separately-authorized `toInstructorReviewDTO()/toAdminDTO()` mapping exists only for the specific endpoints in §2.15/§2.16 that legitimately need broader visibility, and even those never include the raw `ground_truth_definition` seed material outside a dedicated Platform-Admin-only debug endpoint used for scenario-authoring QA (§12.6).

18.4 Session Service — Orchestration Workflow

POST /sessions (§16.4): validate the requested `scenarioId/cohortAssignmentId` (attempt-limit check against `cohort_scenario_assignments`, §6.6) → create the `investigation_sessions` row with a freshly generated `seed` (§6.9) and `status = 'provisioning'` → check the pre-generated telemetry pool (§7.7) for the scenario's current version; if a bundle is available, bulk-load it (bulk INSERTs inside one transaction, remapping bundle-relative IDs to fresh UUIDs) and immediately set `status = 'active'`; if not, enqueue a `telemetry-generation` job (§5.9) and return `provisioning`, with the worker flipping status to `active` and emitting a `session.ready` WebSocket event on completion. POST /sessions/{id}/submit locks the session (`status = 'submitted'`, rejecting further evidence/note/action writes with 409) and enqueues a `scoring` job chained to also trigger the achievements-evaluation step (§13.3) and, if the session is cohort-assigned, notifies the Instructor review queue (§16.13).

18.5 Learning Platform Service — Progress Computation

As established in §13.2, progress is computed on read, not stored: `GET /learning/paths/{id}` joins the caller's `investigation_sessions+scores` against `learning_path_scenarios` for the requested path, cached in Redis per `(userId, pathId)` with a short TTL and explicitly invalidated on session submission (§18.4's submit flow publishes a `progress:invalidate:{userId}` event the Learning Platform Service subscribes to) — a lightweight pub/sub-triggered cache invalidation rather than a cross-service database write, keeping the two services' data ownership boundaries (§18.1) clean.

18.6 Error Handling Strategy

- **Validation errors** (400) are raised at the API Gateway edge (§5.2) wherever possible, before any service code runs, using the same schema as the OpenAPI contract (§16.1) — a single source of truth for "what does a valid request look like," rather than duplicated validation logic per service.
- **Business-rule errors** (409 conflicts like `INSUFFICIENT_EVIDENCE`, §16.6) are raised by domain services as typed exceptions caught by a shared error-mapping middleware that converts them to the standard error envelope (§16.1) — services throw domain-meaningful exceptions (`InsufficientEvidenceError`), never construct HTTP responses directly, keeping business logic testable independent of the HTTP layer.
- **Unexpected errors** (500) are caught by the same middleware, logged with full stack trace + correlation ID server-side, and returned to the client as a generic message with only the correlation ID exposed (never a stack trace or internal detail) — support/debugging uses the correlation ID to find the full server-side log entry (§3.9).
- **Background job failures** (§5.9): BullMQ's built-in retry-with-backoff (default 3 attempts, exponential backoff) handles transient failures (a momentary DB connection blip); a job that exhausts retries moves to a dead-letter queue with alerting (§3.10) rather than being silently dropped, since a permanently-failed `telemetry-generation` job means a Student is stuck on a `provisioning` session and needs either automatic recovery (retry from a pre-generated bundle fallback, §7.7) or a visible, honest error state in the UI rather than an infinite spinner.

18.7 Scheduling and Scaling Strategy Per Service

- **Stateless request-path services** (Identity/Auth, Scenario/Content, Session, Investigation, all three Portal services, Learning Platform, Instructor, Admin) scale horizontally by adding replicas behind the load balancer (§5.15) — no service holds in-memory state that would break under multiple replicas, since all session/cache state lives in Redis/Postgres (§5.6), which is precisely why this constraint was established early in §3.3.
- **Worker services** (Telemetry Generator, Alert Engine, Scoring Engine, Reporting) scale horizontally by adding BullMQ worker processes consuming from their named queues (§5.9); concurrency-per-worker is capped per queue based on the resource profile of that job type (telemetry generation is CPU-heavy and capped low per worker at MVP's shared-VPS resource ceiling; email/reporting jobs are I/O-bound and can run at higher concurrency per worker).
- **Realtime/Notification module** scales horizontally because Redis pub/sub (§5.10), not in-process broadcast, is the fan-out mechanism — any replica can deliver a message to any locally-connected socket regardless of which replica received the originating publish.

- The Scale-phase migration (§19.4) applies Kubernetes Horizontal Pod Autoscaling to each of these independently, using CPU utilization for the Telemetry Generator worker pool specifically (its bottleneck, §7.4) and request-latency/queue-depth-based custom metrics for the request-path services and the other worker pools respectively — i.e., the autoscaling *signal* is chosen per service to match its actual bottleneck, not a single blanket CPU-percentage rule applied uniformly.

19. Deployment Strategy

19.1 Phase 1: Single VPS (MVP)

A single Linux VPS (sizing guidance in §19.2) runs the entire stack via Docker Compose:

```
docker-compose.yml services:
  nginx      - reverse proxy, TLS termination (Let's Encrypt/certbot), static asset serving
  app        - the modular-monolith API Gateway + all domain services (§4.6), N=1-2 replicas
  worker     - background worker process(es) consuming all BullMQ queues (§5.9)
  postgres   - PostgreSQL 15, single instance, volume-mounted for persistence
  redis      - Redis 7, single instance
  minio      - S3-compatible object storage
  prometheus, grafana, loki - observability stack (§5.16)
```

This is the entirety of the production topology for Phase 1. Deployment is a `git pull + docker compose pull && docker compose up -d` on the VPS driven by a CI/CD pipeline (§5.1) that builds and pushes images on merge to `main`, then SSHes into the VPS to trigger the pull/restart — deliberately simple, appropriate for the team size and traffic level this phase is scoped for (§4.6, §20). Zero-downtime deploys are achieved via Nginx + a brief rolling restart of the `app` container behind a health-check gate (§3.7's `/health/ready` endpoints) rather than a full blue-green setup, which is not yet justified at this scale.

19.2 Phase 1 Sizing

Target: the 500-concurrent-session load in §3.3. Recommended starting point: an 8 vCPU / 32GB RAM VPS (a mid-tier offering from any major VPS provider — Hetzner, DigitalOcean, Linode-equivalent), with Postgres and the application/worker processes co-located initially and load-tested before general-availability launch (§3.1's latency targets are the pass/fail criterion for this sizing, not a guess); object storage (MinIO) can run on the same host at this scale given education-content asset sizes are modest (no video hosting in-platform — any produced video content is hosted on a third-party platform and embedded, keeping storage/bandwidth costs off the core infrastructure entirely). Vertical scaling (a larger VPS) is the correct first lever if load testing shows headroom is needed before Phase 2's traffic level justifies horizontal/managed infrastructure investment.

19.3 Phase 2: Managed Data Services, Same Compute Topology

As paid usage grows (§1.7 Phase 2) and before the single-VPS's availability ceiling (§3.2's 99.0% MVP target) becomes a competitive/contractual liability for institutional buyers, the first migration step is **not** a full Kubernetes cut-over — it's swapping the highest-risk self-hosted stateful components for managed equivalents while keeping the same Docker Compose-deployed application containers:

- Self-hosted Postgres → managed Postgres (automated backups, point-in-time recovery, read replica option) — a `DATABASE_URL` environment variable change (§5.18), no code change, because of the repository-layer abstraction in §5.6.
- Self-hosted MinIO → managed S3-compatible object storage (AWS S3, Cloudflare R2, Backblaze B2) — again a config/credential change only (§5.11).

- Self-hosted Redis optionally → managed Redis, if operational burden (not raw cost) becomes the deciding factor.

This step meaningfully improves the DR posture (§3.11's RPO/RTO tighten toward the Scale-phase targets) and availability (no longer a single point of failure on the compute host for data durability) at a modest, usage-justified cost increase, while deliberately deferring the larger architectural/operational investment of Kubernetes until traffic actually demands horizontal scaling beyond what a bigger single VPS can provide.

19.4 Phase 3: Kubernetes, Autoscaling, High Availability

Full migration to a managed Kubernetes service (EKS/GKE/AKS-equivalent):

- Each domain service in §18.1 becomes its own Deployment with its own container image, resource requests/limits, and independent Horizontal Pod Autoscaler configuration (§18.7) — the physical split enabled by, and only by, the module-boundary discipline maintained since MVP (§4.6).
- The API Gateway becomes a Deployment behind a cloud Load Balancer / Ingress controller (§5.15), replacing Nginx's single-VPS role; Nginx's TLS-termination/reverse-proxy role is taken over by the Ingress controller.
- Worker pools become their own Deployments, scaled per §18.7's per-queue signal.
- Postgres runs multi-AZ with a standby (managed service feature); Redis runs as a managed cluster with the three logical roles (cache/pub-sub/queue-broker, §4.2) optionally split onto separate instances once their independent load profiles justify it.
- A service mesh or the platform's native internal networking provides service-to-service TLS and internal DNS-based service discovery (§4.5, §15.3), replacing in-process module calls.
- CI/CD extends to build/push per-service images and apply Kubernetes manifests (Helm charts or equivalent) per environment, with staged rollout (canary or rolling-update with automated health-check-gated rollback) replacing the simple restart-based deploy of Phase 1.

19.5 Disaster Recovery Runbook (Summary)

A documented, versioned runbook (kept in the repository, not only in an external wiki, so it stays in sync with the actual deployed topology) covers, at minimum: (1) full-stack restore from the most recent Postgres backup + WAL replay onto a fresh host, (2) object-storage restore from its independently-replicated backup (§3.11), (3) DNS/TLS cert re-issuance if the restore target is a new host, (4) post-restore validation checklist (health checks, a synthetic end-to-end scenario-session smoke test) before declaring recovery complete, and (5) a quarterly scheduled drill (restoring into an isolated environment, not production) to keep the runbook's RTO claim (§3.11) honest rather than aspirational.

19.6 CDN and Static Asset Delivery Across Phases

Consistent across all three phases (§5.14): the built frontend bundle and public marketing/catalog pages are served through a CDN from day one, decoupled from the backend compute topology entirely — this is the one piece of infrastructure that does not need to "migrate" between phases, since a CDN in front of a static asset origin is equally valid whether that origin is the Phase 1 VPS's Nginx, Phase 2's object storage bucket, or Phase 3's Kubernetes-served build artifact.

19.7 Environment Parity

Local development uses the identical `docker-compose.yml` (with a `docker-compose.override.yml` for dev-only conveniences like hot-reload volume mounts and seeded test data) as Phase 1 production, and CI runs integration tests against the same containers — the explicit goal (§3.7) being that "works in CI" and "works in production" are backed by the same container images and the same data-store technology at every phase, so environment-specific bugs are a rare, actively-designed-against category rather than a routine occurrence.

20. Cost Optimization

20.1 Guiding Principle

Every infrastructure decision in this document was made against a single constraint, stated explicitly here because it explains *why* the choices in §5 and §19 look the way they do: **the platform must run a commercially credible SaaS product on a single, modestly-sized VPS at MVP, with no infrastructure line item that scales with usage before the product has paying usage to scale against.** Cost optimization is not a separate afterthought layered on top of the architecture — it is one of the reasons the architecture looks like it does (the modular monolith of §4.6, the "generate once, archive after 90 days" telemetry lifecycle of §6.23, the deliberate exclusion of any per-session live LLM call from the core product in §7.6).

20.2 Cost Breakdown by Category (Phase 1 / MVP)

Category	Choice	Approx. cost driver	Why this minimizes cost
Compute	One 8 vCPU/32GB VPS (§19.2)	Fixed monthly, provider-dependent, low three figures/month at this tier	One host instead of N managed services each with their own baseline fee; vertical scaling (bigger VPS) is cheaper per unit of headroom than horizontal managed infrastructure at this traffic level
Database	Self-hosted Postgres on the same VPS	\$0 beyond the VPS itself	Avoids a managed-Postgres monthly minimum before there's revenue to justify it (§19.3 explicitly defers this)
Cache/Queue	Self-hosted Redis, same VPS	\$0 beyond the VPS	One process serving three roles (§4.2) instead of three separate paid services
Object Storage	Self-hosted MinIO, same VPS	\$0 beyond the VPS/attached disk	No per-GB/per-request billing until traffic genuinely requires offloading storage from the compute host
CDN	Free-tier CDN (§5.14)	\$0	Free tiers at major CDN providers comfortably cover MVP-level static asset traffic
Email delivery	Transactional email provider's free/starter tier	\$0–low monthly	Cheaper and far more reliable than self-hosting SMTP and managing deliverability/reputation from scratch (§5.9)

Category	Choice	Approx. cost driver	Why this minimizes cost
Observability	Self-hosted Prometheus/Grafana/Loki, same VPS	\$0 beyond the VPS	Avoids a managed observability vendor's per-host/per-GB-ingested pricing before scale justifies it
TLS	Let's Encrypt (§19.1)	\$0	Free, automated, industry-standard
CI/CD	GitHub Actions free tier	\$0 at MVP volume	Sufficient minutes/month for a small team's build/deploy cadence
AI features	Not built at MVP (§14.8)	\$0	Deferred entirely to Phase 3, avoiding per-call LLM API cost before there's subscription revenue to fund it
Payments	Third-party processor, transaction-fee-only pricing	Percentage of revenue, not a fixed cost	No fixed monthly fee; cost scales naturally with revenue (§15.8)

The realistic, honest takeaway: Phase 1 infrastructure cost is dominated almost entirely by the single VPS line item, everything else is free-tier or effectively free at this traffic level, and the single largest non-infrastructure cost at this stage is the team's own time — which is exactly why the modular-monolith/single-deployable-artifact choice (§4.6) matters as much for cost as the hosting bill does: it minimizes the operational time a small team spends on deployment/ops toil.

20.3 Deliberate Cost/Quality Trade-offs Made Explicit

- **99.0% availability, not 99.9%, at MVP (§3.2).** A multi-AZ, multi-replica setup capable of 99.9%+ costs meaningfully more (managed DB with standby, multiple compute hosts, a real load balancer tier) and is not commercially necessary for an education product's MVP phase; this is stated as an honest trade-off, not hidden from the availability target.
- **24-hour RPO, not near-zero, at MVP (§3.11).** Continuous point-in-time backup/replication is a paid managed-database feature; nightly backup + WAL archiving gets most of the practical benefit at a fraction of the cost, with the gap closed at Phase 2 once revenue justifies it (§19.3).
- **No live LLM-powered features at MVP (§14.8).** Per-call AI API cost, especially at a per-session granularity across hundreds of concurrent sessions, is one of the few costs in this stack that scales directly with usage rather than being a fixed monthly fee — exactly the kind of cost this phase is designed to avoid until subscription revenue exists to fund it, hence AI features are fully specified but explicitly deferred (§14.1).
- **Synthetic content generation has zero third-party API cost (§7.6).** By construction, the single most frequently-executed operation in the product (every scenario start, §7.7) has no external API

dependency and therefore no per-use marginal cost beyond the VPS's own compute — this was a cost-driven architectural requirement, not only a latency/reliability one.

- **No video hosting infrastructure (§19.2).** Bandwidth-heavy content types are explicitly kept off the core infrastructure and delegated to third-party platforms built for that purpose, avoiding a cost category (egress bandwidth for video) that scales badly on a single VPS.

20.4 What Triggers Each Spend Increase

Rather than a fixed calendar-based upgrade schedule, every infrastructure spend increase in this document is tied to a concrete, observable trigger, so money is spent when the product's growth actually demands it, not speculatively:

Spend increase	Trigger
Bigger VPS (vertical scaling)	Load testing or production monitoring (§3.10) shows §3.1 latency targets are at risk under current concurrent-session volume
Managed Postgres/object storage (§19.3)	First institutional/enterprise customer with a contractual DR/availability requirement the single-VPS setup can't credibly meet, or database size/load approaching what a single host can comfortably serve
Kubernetes migration (§19.4)	Concurrent-session volume consistently exceeds what vertical scaling on a single host can absorb, and revenue justifies the added operational complexity/cost of a multi-service, multi-node deployment
AI features (§14)	Subscription revenue run-rate can absorb ongoing per-call LLM API cost as a normal COGS line item without threatening unit economics
Managed observability vendor	Self-hosted Prometheus/Grafana/Loki's own resource footprint starts meaningfully competing with application workload for VPS capacity

This trigger-based framing is the practical answer to "how do we keep the MVP lean while leaving room to scale" (§1.7, §3.6): every later-phase capability described in §19.3–§19.4 is fully specified in this document *now*, so no future spend decision requires new architectural design work — it only requires recognizing that its trigger condition has been met.